

循序渐进，学习开发 xv6-riscv

第 15 章 进程

汪辰



目录

01

为进程创建用户态页表

02

用户态 Trap

03

第一个用户态程序/进程

04

改进程序构建



01

为进程创建用户态页表

为进程创建用户态页表

```
diff --git a/kernel/proc.h b/kernel/proc.h
index ed2df79..7353a77 100644
--- a/kernel/proc.h
+++ b/kernel/proc.h
@@ -40,6 +40,8 @@ struct proc {
    // these are private to the process, so p->lock need not be held.
    uint64 kstack;           // Virtual address of kernel stack
+   uint64 sz;               // Size of process memory (bytes)
+   pagetable_t pagetable;   // User page table
    struct context context;   // swtch() here to run process
    char name[16];           // Process name (debugging)
    void (*start)(void);     // Function to run when process starts
```

为进程创建用户态页表

```
diff --git a/kernel/proc.h b/kernel/proc.h
index ed2df79..7353a77 100644
--- a/kernel/proc.h
+++ b/kernel/proc.h
@@ -40,6 +40,8 @@ struct proc {
    // these are private to the process, so p->lock need not be held.
    uint64 kstack;           // Virtual address of kernel stack
+   uint64 sz;               // Size of process memory (bytes)
+   pagetable_t pagetable;   // User page table
    struct context context;   // swtch() here to run process
    char name[16];           // Process name (debugging)
    void (*start)(void);     // Function to run when process starts
```

```
static struct proc* allocproc(void (*start_routine)(void))
{
    struct proc *p;
    // .....
    found:
    p->pid = allocpid();
    p->state = USED;

    // An empty user page table.
    p->pagetable = proc_pagetable(p);
    if(p->pagetable == 0){
        freeproc(p);
        release(&p->lock);
        return 0;
    }

    // Set up new context to start executing
    //.....
    return p;
}
```

为进程创建用户态页表

```
diff --git a/kernel/proc.h b/kernel/proc.h
index ed2df79..7353a77 100644
--- a/kernel/proc.h
+++ b/kernel/proc.h
@@ -40,6 +40,8 @@ struct proc {
    // these are private to the process, so p->lock need not be held.
    uint64 kstack;           // Virtual address of kernel stack
+   uint64 sz;               // Size of process memory (bytes)
+   pagetable_t pagetable;   // User page table
    struct context context;   // switch() here to run process
    char name[16];           // Process name (debugging)
    void (*start)(void);     // Function to run when process starts
```

```
static struct proc* allocproc(void (*start_routine)(void))
{
    struct proc *p;
    // .....
    found:
    p->pid = allocpid();
    p->state = USED;

    // An empty user page table.
    p->pagetable = proc_pagetable(p);
    if(p->pagetable == 0){
        freeproc(p);
        release(&p->lock);
        return 0;
    }

    // Set up new context to start executing
    // .....
    return p;
}
```

```
pagetable_t proc_pagetable(struct proc *p)
{
    pagetable_t pagetable;
    // An empty page table.
    pagetable = uvmcreate();
    if(pagetable == 0)
        return 0;
    return pagetable;
}
```

```
static void freeproc(struct proc *p)
{
    if(p->pagetable)
        proc_freepagetable(p->pagetable, p->sz);
    p->pagetable = 0;
    p->sz = 0;
    p->pid = 0;
    p->name[0] = 0;
    p->state = UNUSED;
}
```

```
void proc_freepagetable(pagetable_t pagetable, uint64 sz)
{
    uvmfree(pagetable, sz);
}
```

为进程创建用户态页表

```
diff --git a/kernel/proc.h b/kernel/proc.h
index ed2df79..7353a77 100644
--- a/kernel/proc.h
+++ b/kernel/proc.h
@@ -40,6 +40,8 @@ struct proc {
    // these are private to the process, so p->lock need not be held.
    uint64 kstack;           // Virtual address of kernel stack
+   uint64 sz;               // Size of process memory (bytes)
+   pagetable_t pagetable;   // User page table
    struct context context;  // switch() here to run process
    char name[16];           // Process name (debugging)
    void (*start)(void);     // Function to run when process starts
```

```
static struct proc* allocproc(void (*start_routine)(void))
{
    struct proc *p;
    // .....
    found:
    p->pid = allocpid();
    p->state = USED;

    // An empty user page table.
    p->pagetable = proc_pagetable(p);
    if(p->pagetable == 0){
        freeproc(p);
        release(&p->lock);
        return 0;
    }

    // Set up new context to start executing
    // .....
    return p;
}
```

```
pagetable_t proc_pagetable(struct proc *p)
{
    pagetable_t pagetable;
    // An empty page table.
    pagetable = uvmcreate();
    if(pagetable == 0)
        return 0;
    return pagetable;
}
```

```
static void freeproc(struct proc *p)
{
    if(p->pagetable)
        proc_freepagetable(p->pagetable, p->sz);
    p->pagetable = 0;
    p->sz = 0;
    p->pid = 0;
    p->name[0] = 0;
    p->state = UNUSED;

    void proc_freepagetable(pagetable_t pagetable, uint64 sz)
    {
        uvmfree(pagetable, sz);
    }
}
```

为进程创建用户态页表

```
pagetable_t uvmcreate()
{
    pagetable_t pagetable;
    pagetable = (pagetable_t) kalloc();
    if(pagetable == 0)
        return 0;
    memset(pagetable, 0, PGSIZE);
    return pagetable;
}
```

```
void uvmfree(pagetable_t pagetable, uint64 sz)
{
    if(sz > 0)
        uvmunmap(pagetable, 0, PGROUNDUP(sz)/PGSIZE, 1);
    freewalk(pagetable);
}
```

为进程创建用户态页表

```
pagetable_t uvmcreate()
{
    pagetable_t pagetable;
    pagetable = (pagetable_t) kalloc();
    if(pagetable == 0)
        return 0;
    memset(pagetable, 0, PGSIZE);
    return pagetable;
}
```

```
void uvmfree(pagetable_t pagetable, uint64 sz)
{
    if(sz > 0)
        uvmunmap(pagetable, 0, PGROUNDUP(sz)/PGSIZE, 1);
    freewalk(pagetable);
}
```

```
void uvmunmap(pagetable_t pagetable,
              uint64 va, uint64 npages, int do_free)
{
    uint64 a;
    pte_t *pte;

    if((va % PGSIZE) != 0)
        panic("uvmunmap: not aligned");

    for(a = va; a < va + npages*PGSIZE; a += PGSIZE){
        if((pte = walk(pagetable, a, 0)) == 0)
            panic("uvmunmap: walk");
        if((*pte & PTE_V) == 0)
            panic("uvmunmap: not mapped");
        if(PTE_FLAGS(*pte) == PTE_V)
            panic("uvmunmap: not a leaf");
        if(do_free){
            uint64 pa = PTE2PA(*pte);
            kfree((void*)pa);
        }
        *pte = 0;
    }
}
```

为进程创建用户态页表

```
pagetable_t uvmcreate()
{
    pagetable_t pagetable;
    pagetable = (pagetable_t) kalloc();
    if(pagetable == 0)
        return 0;
    memset(pagetable, 0, PGSIZE);
    return pagetable;
}
```

```
void uvmfree(pagetable_t pagetable, uint64 sz)
{
    if(sz > 0)
        uvmunmap(pagetable, 0, PGROUNDUP(sz)/PGSIZE, 1);
    freewalk(pagetable);
}
```

```
void uvmunmap(pagetable_t pagetable,
              uint64 va, uint64 npages, int do_free)
{
    uint64 a;
    pte_t *pte;

    if((va % PGSIZE) != 0)
        panic("uvmunmap: not aligned");

    for(a = va; a < va + npages*PGSIZE; a += PGSIZE){
        if((pte = walk(pagetable, a, 0)) == 0)
            panic("uvmunmap: walk");
        if((*pte & PTE_V) == 0)
            panic("uvmunmap: not mapped");
        if(PTE_FLAGS(*pte) == PTE_V)
            panic("uvmunmap: not a leaf");
        if(do_free){
            uint64 pa = PTE2PA(*pte);
            kfree((void*)pa);
        }
        *pte = 0;
    }
}
```

- pagetable: page table
- va: virtual address
- npages:要取消映射的页面数量
- do_free:是否释放物理内存 (1:释放, 0:仅取消映射)

对以 va 为起始地址的连续的 npages 个 PGSIZE 的地址空间取消映射。如果 do_free 不为 0, 则还会释放 PTE 对应的物理页的内存空间。

为进程创建用户态页表

```
pagetable_t uvmcreate()
{
    pagetable_t pagetable;
    pagetable = (pagetable_t) kalloc();
    if(pagetable == 0)
        return 0;
    memset(pagetable, 0, PGSIZE);
    return pagetable;
}
```

```
void uvmfree(pagetable_t pagetable, uint64 sz)
{
    if(sz > 0)
        uvmunmap(pagetable, 0, PGROUNDUP(sz)/PGSIZE, 1);
    freewalk(pagetable);
}
```

```
void uvmunmap(pagetable_t pagetable,
              uint64 va, uint64 npages, int do_free)
{
    uint64 a;
    pte_t *pte;

    if((va % PGSIZE) != 0)
        panic("uvmunmap: not aligned");

    for(a = va; a < va + npages*PGSIZE; a += PGSIZE){
        if((pte = walk(pagetable, a, 0)) == 0)
            panic("uvmunmap: walk");
        if(*pte & PTE_V == 0)
            panic("uvmunmap: not mapped");
        if(PTE_FLAGS(*pte) == PTE_V)
            panic("uvmunmap: not a leaf");
        if(do_free){
            uint64 pa = PTE2PA(*pte);
            kfree((void*)pa);
        }
        *pte = 0;
    }
}
```

- pagetable: page table
- va: virtual address
- npages:要取消映射的页面数量
- do_free:是否释放物理内存 (1:释放, 0:仅取消映射)

对以 va 为起始地址的连续的 npages 个 PGSIZE 的地址空间取消映射。如果 do_free 不为 0, 则还会释放 PTE 对应的物理页的内存空间。

调用 walk 查找 PTE。第三个参数 0 表示不分配新页表, 只读查找。

为进程创建用户态页表

```
pagetable_t uvmcreate()
{
    pagetable_t pagetable;
    pagetable = (pagetable_t) kalloc();
    if(pagetable == 0)
        return 0;
    memset(pagetable, 0, PGSIZE);
    return pagetable;
}
```

```
void uvmfree(pagetable_t pagetable, uint64 sz)
{
    if(sz > 0)
        uvmunmap(pagetable, 0, PGROUNDUP(sz)/PGSIZE, 1);
    freewalk(pagetable);
}
```

```
void uvmunmap(pagetable_t pagetable,
              uint64 va, uint64 npages, int do_free)
{
    uint64 a;
    pte_t *pte;

    if((va % PGSIZE) != 0)
        panic("uvmunmap: not aligned");

    for(a = va; a < va + npages*PGSIZE; a += PGSIZE){
        if((pte = walk(pagetable, a, 0)) == 0)
            panic("uvmunmap: walk");
        if((*pte & PTE_V) == 0)
            panic("uvmunmap: not mapped");
        if(PTE_FLAGS(*pte) == PTE_V)
            panic("uvmunmap: not a leaf");
        if(do_free){
            uint64 pa = PTE2PA(*pte);
            kfree((void*)pa);
        }
        *pte = 0;
    }
}
```

- pagetable: page table
- va: virtual address
- npages:要取消映射的页面数量
- do_free:是否释放物理内存 (1:释放, 0:仅取消映射)

对以 va 为起始地址的连续的 npages 个 PGSIZE 的地址空间取消映射。如果 do_free 不为 0, 则还会释放 PTE 对应的物理页的内存空间。

调用 walk 查找 PTE。第三个参数 0 表示不分配新页表, 只读查找。

- 一系列检查:
- walk 必须找到
 - 找到的 PTE 必须对应一个有效映射
 - 找到的 PTE 必须是一个 LO 的

为进程创建用户态页表

```
pagetable_t uvmcreate()
{
    pagetable_t pagetable;
    pagetable = (pagetable_t) kalloc();
    if(pagetable == 0)
        return 0;
    memset(pagetable, 0, PGSIZE);
    return pagetable;
}
```

```
void uvmfree(pagetable_t pagetable, uint64 sz)
{
    if(sz > 0)
        uvmunmap(pagetable, 0, PGROUNDUP(sz)/PGSIZE, 1);
    freewalk(pagetable);
}
```

```
void uvmunmap(pagetable_t pagetable,
              uint64 va, uint64 npages, int do_free)
{
    uint64 a;
    pte_t *pte;

    if((va % PGSIZE) != 0)
        panic("uvmunmap: not aligned");

    for(a = va; a < va + npages*PGSIZE; a += PGSIZE){
        if((pte = walk(pagetable, a, 0)) == 0)
            panic("uvmunmap: walk");
        if(*pte & PTE_V == 0)
            panic("uvmunmap: not mapped");
        if(PTE_FLAGS(*pte) == PTE_V)
            panic("uvmunmap: not a leaf");
        if(do_free){
            uint64 pa = PTE2PA(*pte);
            kfree((void*)pa);
        }
        *pte = 0;
    }
}
```

- pagetable: page table
- va: virtual address
- npages:要取消映射的页面数量
- do_free:是否释放物理内存 (1:释放, 0:仅取消映射)

对以 va 为起始地址的连续的 npages 个 PGSIZE 的地址空间取消映射。如果 do_free 不为 0, 则还会释放 PTE 对应的物理页的内存空间。

调用 walk 查找 PTE。第三个参数 0 表示不分配新页表, 只读查找。

- 一系列检查:
- walk 必须找到
 - 找到的 PTE 必须对应一个有效映射
 - 找到的 PTE 必须是一个 LO 的

如果 do_free 不为 0 则执行释放叶子页帧的操作。

为进程创建用户态页表

```
pagetable_t uvmcreate()
{
    pagetable_t pagetable;
    pagetable = (pagetable_t) kalloc();
    if(pagetable == 0)
        return 0;
    memset(pagetable, 0, PGSIZE);
    return pagetable;
}
```

```
void uvmfree(pagetable_t pagetable, uint64 sz)
{
    if(sz > 0)
        uvmunmap(pagetable, 0, PGROUNDUP(sz)/PGSIZE, 1);
    freewalk(pagetable);
}
```

```
void uvmunmap(pagetable_t pagetable,
              uint64 va, uint64 npages, int do_free)
{
    uint64 a;
    pte_t *pte;

    if((va % PGSIZE) != 0)
        panic("uvmunmap: not aligned");

    for(a = va; a < va + npages*PGSIZE; a += PGSIZE){
        if((pte = walk(pagetable, a, 0)) == 0)
            panic("uvmunmap: walk");
        if(pte & PTE_V)
            uvmfree(pagetable, a, 1);
    }
```

```
void freewalk(pagetable_t pagetable)
{
    // there are 2^9 = 512 PTEs in a page table.
    for(int i = 0; i < 512; i++){
        pte_t pte = pagetable[i];
        if((pte & PTE_V) && (pte & (PTE_R|PTE_W|PTE_X)) == 0){
            // this PTE points to a lower-level page table.
            uint64 child = PTE2PA(pte);
            freewalk((pagetable_t)child);
            pagetable[i] = 0;
        } else if(pte & PTE_V){
            panic("freewalk: leaf");
        }
    }
    kfree((void*)pagetable);
}
```

递归释放页表页（page-table pages）本身占用的内存，前提是所有叶子映射（leaf mappings）已经被提前移除（在 uvmfree() 中移除动作已经在 uvmunmap() 中完成）。



02

用户态 Trap



用户态 Trap

- 内核态 Trap
- 用户态 Trap



用户态 Trap

- **内核态 Trap**
- 用户态 Trap

```
.globl kerneltrap
.globl kernelvec
.align 4
kernelvec:
    # make room to save registers.
    addi sp, sp, -256

    # save caller-saved registers.
    sd ra, 0(sp)
    # sd sp, 8(sp)
    sd gp, 16(sp)
    .....
    sd t6, 240(sp)

    # call the C trap handler in trap.c
    call kerneltrap

    # restore registers.
    ld ra, 0(sp)
    # ld sp, 8(sp)
    ld gp, 16(sp)
    .....
    ld t6, 240(sp)

    addi sp, sp, 256

    # return to whatever we were doing in the kernel.
    sret
```

用户态 Trap

- 内核态 Trap

- **用户态 Trap**

```
uservec:
    # save the user registers in TRAPFRAME
    .....
    # call usertrap()
    .....
    # restore all from TRAPFRAME
    .....
    sret
```

用户态 Trap

- 内核态 Trap
- **用户态 Trap**

```
uservec:
    # save the user registers in TRAPFRAME
    .....
    # call usertrap()
    .....
    # restore all from TRAPFRAME
    .....
    sret
```

```
struct trapframe {
/* 0 */ uint64 kernel_satp; // kernel page table
/* 8 */ uint64 kernel_sp; // top of process's kernel stack
/* 16 */ uint64 kernel_trap; // usertrap()
/* 24 */ uint64 epc; // saved user program counter
/* 32 */ uint64 kernel_hartid; // saved kernel tp
/* 40 */ uint64 ra;
/* 48 */ uint64 sp;
/* 56 */ uint64 gp;
/* 64 */ uint64 tp;
/* 72 */ uint64 t0;
/* 80 */ uint64 t1;
/* 88 */ uint64 t2;
/* 96 */ uint64 s0;
/* 104 */ uint64 s1;
/* 112 */ uint64 a0;
/* 120 */ uint64 a1;
/* 128 */ uint64 a2;
/* 136 */ uint64 a3;
/* 144 */ uint64 a4;
/* 152 */ uint64 a5;
/* 160 */ uint64 a6;
/* 168 */ uint64 a7;
/* 176 */ uint64 s2;
/* 184 */ uint64 s3;
/* 192 */ uint64 s4;
/* 200 */ uint64 s5;
/* 208 */ uint64 s6;
/* 216 */ uint64 s7;
/* 224 */ uint64 s8;
/* 232 */ uint64 s9;
/* 240 */ uint64 s10;
/* 248 */ uint64 s11;
/* 256 */ uint64 t3;
/* 264 */ uint64 t4;
/* 272 */ uint64 t5;
/* 280 */ uint64 t6;
};
```


用户态 Trap

- 内核态 Trap
- 用户态 Trap

```
uservec:
    # save the user registers in TRAPFRAME
    .....
    # call usertrap()
    .....
    # restore all from TRAPFRAME
    .....
    sret
```

```
struct trapframe {
/* 0 */ uint64 kernel_satp; // kernel page table
/* 8 */ uint64 kernel_sp; // top
/* 16 */ uint64 kernel_trap; // user
/* 24 */ uint64 epc; // save
/* 32 */ uint64 kernel_hartid; // save
/* 40 */ uint64 ra;
/* 48 */ uint64 sp;
/* 56 */ uint64 gp;
/* 64 */ uint64 tp;
/* 72 */ uint64 t0;
/* 80 */ uint64 t1;
/* 88 */ uint64 t2;
/* 96 */ uint64 s0;
/* 104 */ uint64 s1;
/* 112 */ uint64 a0;
/* 120 */ uint64 a1;
/* 128 */ uint64 a2;
/* 136 */ uint64 a3;
/* 144 */ uint64 a4;
/* 152 */ uint64 a5;
/* 160 */ uint64 a6;
/* 168 */ uint64 a7;
/* 176 */ uint64 s2;
/* 184 */ uint64 s3;
/* 192 */ uint64 s4;
/* 200 */ uint64 s5;
/* 208 */ uint64 s6;
/* 216 */ uint64 s7;
/* 224 */ uint64 s8;
/* 232 */ uint64 s9;
/* 240 */ uint64 s10;
/* 248 */ uint64 s11;
/* 256 */ uint64 t3;
/* 264 */ uint64 t4;
/* 272 */ uint64 t5;
/* 280 */ uint64 t6;
};

struct proc {
    // .....
    struct trapframe *trapframe; // data page for trampoline.S
    // .....
};

static struct proc* allocproc(void (*start_routine)(void))
{
    struct proc *p;
    // .....
found:
    p->pid = allocpid();
    p->state = USED;

    // Allocate a trapframe page.
    if((p->trapframe = (struct trapframe *)kalloc()) == 0){
        freeproc(p);
        release(&p->lock);
        return 0;
    }
    // .....
    return p;
}

// free a proc structure and the data hanging from it,
// including user pages.
// p->lock must be held.
static void freeproc(struct proc *p)
{
    if(p->trapframe)
        kfree((void*)p->trapframe);
    p->trapframe = 0;
    // .....
}
```

用户态 Trap

- 内核态 Trap
- 用户态 Trap

```
uservec:
    # save the user registers in TRAPFRAME
    .....
    # call usertrap()
    .....
    # restore all from TRAPFRAME
    .....
    sret
```

```
struct trapframe {
    /* 0 */ uint64 kernel_satp; // kernel page table
    /* 8 */ uint64 kernel_sp; // top of kernel stack
    /* 16 */ uint64 kernel_trap; // user trapno
    /* 24 */ uint64 epc; // saved user epc
    /* 32 */ uint64 kernel_hartid; // saved kernel hartid
    /* 40 */ uint64 ra; // saved user ra
    /* 48 */ uint64 sp; // saved user stack pointer
    /* 56 */ uint64 gp; // saved user gp
    /* 64 */ uint64 tp; // saved user tp
    /* 72 */ uint64 t0; // saved user t0
    /* 80 */ uint64 t1; // saved user t1
    /* 88 */ uint64 t2; // saved user t2
    /* 96 */ uint64 s0; // saved user s0
    /* 104 */ uint64 s1; // saved user s1
    /* 112 */ uint64 a0; // saved user a0
    /* 120 */ uint64 a1; // saved user a1
    /* 128 */ uint64 a2; // saved user a2
    /* 136 */ uint64 a3; // saved user a3
    /* 144 */ uint64 a4; // saved user a4
    /* 152 */ uint64 a5; // saved user a5
    /* 160 */ uint64 a6; // saved user a6
    /* 168 */ uint64 a7; // saved user a7
    /* 176 */ uint64 s2; // saved user s2
    /* 184 */ uint64 s3; // saved user s3
    /* 192 */ uint64 s4; // saved user s4
    /* 200 */ uint64 s5; // saved user s5
    /* 208 */ uint64 s6; // saved user s6
    /* 216 */ uint64 s7; // saved user s7
    /* 224 */ uint64 s8; // saved user s8
    /* 232 */ uint64 s9; // saved user s9
    /* 240 */ uint64 s10; // saved user s10
    /* 248 */ uint64 s11; // saved user s11
    /* 256 */ uint64 t3; // saved user t3
    /* 264 */ uint64 t4; // saved user t4
    /* 272 */ uint64 t5; // saved user t5
    /* 280 */ uint64 t6; // saved user t6
};

struct proc {
    // .....
    struct trapframe *trapframe; // data page for trampoline.S
    // .....
};

// Allocate a trapframe page.
if((p->trapframe = (struct trapframe *)kalloc()) == 0){
    freeproc(p);
    release(&p->lock);
    return 0;
}

// .....
return p;

// free a proc structure and the data hanging from it,
// including user pages.
// p->lock must be held.
static void freeproc(struct proc *p)
{
    if(p->trapframe)
        kfree((void*)p->trapframe);
    p->trapframe = 0;
    // .....
}
```

- kernel_satp: 用于保存和恢复内核的根页表地址
- kernel_sp: 用于保存和恢复进程内核态的栈顶
- kernel_trap: 用于保存和恢复 usertrap 的地址
- epc: 为了系统调用的处理
- kernel_hartid: 用于保存和恢复 tp

用户态 Trap

- 内核态 Trap
- 用户态 Trap

```
uservec:
    # save the user registers in TRAPFRAME
    .....
    # call usertrap()
    .....
    # restore all from TRAPFRAME
    .....
    sret
```

```
struct trapframe {
    /* 0 */ uint64 kernel_satp; // kernel page table
    /* 8 */ uint64 kernel_sp; // top
    /* 16 */ uint64 kernel_trap; // user
    /* 24 */ uint64 epc; // save
    /* 32 */ uint64 kernel_hartid; // save
    /* 40 */ uint64 ra;
    /* 48 */ uint64 sp;
    /* 56 */ uint64 gp;
    /* 64 */ uint64 tp;
    /* 72 */ uint64 t0;
    /* 80 */ uint64 t1;
    /* 88 */ uint64 t2;
    /* 96 */ uint64 s0;
    /* 104 */ uint64 s1;
    /* 112 */ uint64 a0;
    /* 120 */ uint64 a1;
    /* 128 */ uint64 a2;
    /* 136 */ uint64 a3;
    /* 144 */ uint64 a4;
    /* 152 */ uint64 a5;
    /* 160 */ uint64 a6;
    /* 168 */ uint64 a7;
    /* 176 */ uint64 s2;
    /* 184 */ uint64 s3;
    /* 192 */ uint64 s4;
    /* 200 */ uint64 s5;
    /* 208 */ uint64 s6;
    /* 216 */ uint64 s7;
    /* 224 */ uint64 s8;
    /* 232 */ uint64 s9;
    /* 240 */ uint64 s10;
    /* 248 */ uint64 s11;
    /* 256 */ uint64 t3;
    /* 264 */ uint64 t4;
    /* 272 */ uint64 t5;
    /* 280 */ uint64 t6;
};

struct proc {
    // .....
    struct trapframe *trapframe; // data page for trampoline.S
    // .....
};

static struct proc* allocproc(void (*start_routine)(void))
{
    struct proc *p;
    // .....
found:
    p->pid = allocpid();
    p->state = USED;

    // Allocate a trapframe page.
    if((p->trapframe = (struct trapframe *)kalloc()) == 0){
        freeproc(p);
        release(&p->lock);
        return 0;
    }
    // .....
    return p;
}

// free a proc structure and the data hanging from it,
// including user pages.
// p->lock must be held.
static void freeproc(struct proc *p)
{
    if(p->trapframe)
        kfree((void*)p->trapframe);
    p->trapframe = 0;
    // .....
}
```

用户态 Trap

- 内核态 Trap
- 用户态 Trap

```
uservec:  
    # save the user registers in TRAPFRAME  
    .....  
    # call usertrap()  
    .....  
    # restore all from TRAPFRAME  
    .....  
    sret
```

将用户进程页表切换为内核页表

将内核页表切换回用户进程页表

```
struct trapframe {  
    /* 0 */ uint64 kernel_satp; // kernel page table  
    /* 8 */ uint64 kernel_sp; // top of kernel stack  
    /* 16 */ uint64 kernel_trap; // user trapno  
    /* 24 */ uint64 epc; // saved user epc  
    /* 32 */ uint64 kernel_hartid; // saved user hartid  
    /* 40 */ uint64 ra; // saved user ra  
};
```

```
struct proc {  
    // .....  
    struct trapframe *trapframe; // data page for trampoline.S  
    // .....  
};
```

```
struct proc* allocproc(void (*start_routine)(void))  
{  
    struct proc *p;  
    // .....  
}
```

```
found:  
    p->pid = allocpid();  
    p->state = USED;
```

```
    // allocate a trapframe page.  
    p->trapframe = (struct trapframe *)kalloc() == 0 ?  
        0 : (struct trapframe *)kalloc();  
    proc(p);
```

```
    release(&p->lock);  
    return 0;  
}  
// .....  
return p;  
}
```

```
    // free a proc structure and the data hanging from it,  
    // including user pages.  
    // p->lock must be held.
```

```
static void freeproc(struct proc *p)  
{  
    if(p->trapframe)  
        kfree((void*)p->trapframe);  
    p->trapframe = 0;  
    // .....  
}
```

```
/* 160 */ uint64 a6;  
/* 168 */ uint64 a7;  
/* 176 */ uint64 s2;  
/* 184 */ uint64 s3;  
/* 192 */ uint64 s4;  
/* 200 */ uint64 s5;  
/* 208 */ uint64 s6;  
/* 216 */ uint64 s7;  
/* 224 */ uint64 s8;  
/* 232 */ uint64 s9;  
/* 240 */ uint64 s10;  
/* 248 */ uint64 s11;  
/* 256 */ uint64 t3;  
/* 264 */ uint64 t4;  
/* 272 */ uint64 t5;  
/* 280 */ uint64 t6;  
};
```

用户态 Trap

- 内核态 Trap
- 用户态 Trap

```
uservec:  
# save the user registers in TRAPFRAME  
.....  
# call usertrap()  
.....  
# restore all from TRAPFRAME  
.....  
sret  
MAXVA
```

内核虚拟地址空间

trampoline

TRAMPOLINE

0

将用户进程页表切换为内核页表

将内核页表切换为用户进程页表

物理地址空间

uservec:
.....

uservec
};

```
struct trapframe {  
/* 0 */ uint64 kernel_satp; // kernel page table  
/* 8 */ uint64 kernel_sp; // top  
/* 16 */ uint64 kernel_trap; // user  
/* 24 */ uint64 epc; // save  
/* 32 */ uint64 kernel_hartid; // save  
/* 40 */ uint64 ra;  
};
```

```
struct proc {  
// .....  
struct trapframe *trapframe; // data page for trampoline.S  
// .....  
};
```

```
struct proc* allocproc(void (*start_routine)(void))  
{  
proc *p;  
// .....  
found:  
p->pid = allocpid();  
p->state = USED;
```

```
locate a trapframe page.  
p->trapframe = (struct trapframe *)kalloc() == 0){  
return 0;  
proc(p);  
release(&p->lock);  
return 0;  
}
```

进程虚拟地址空间

trampoline

MAXVA

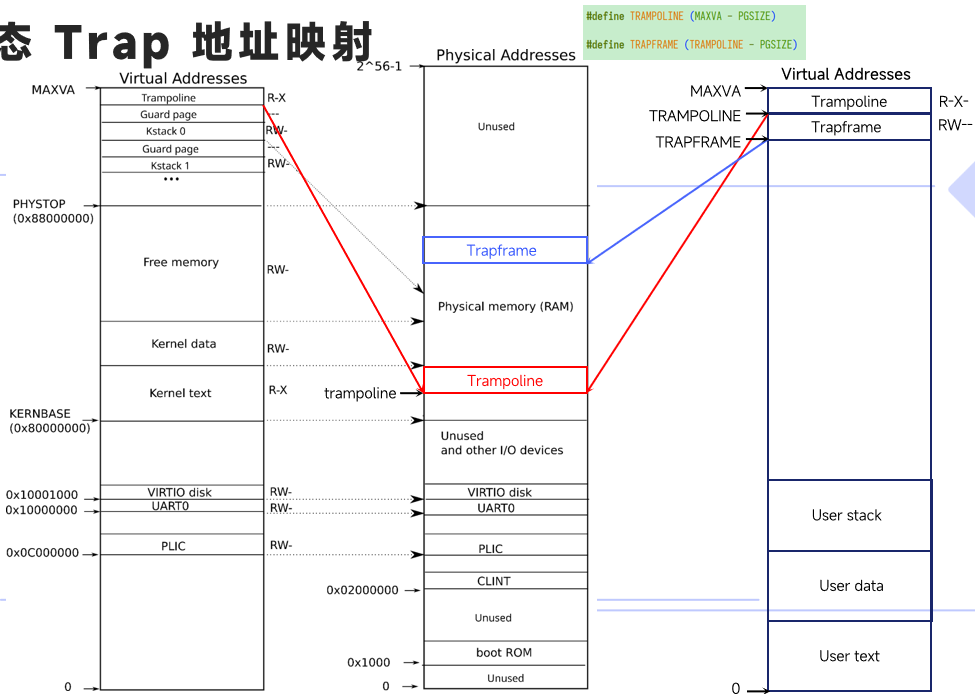
and the data hanging from it,

struct proc *p)

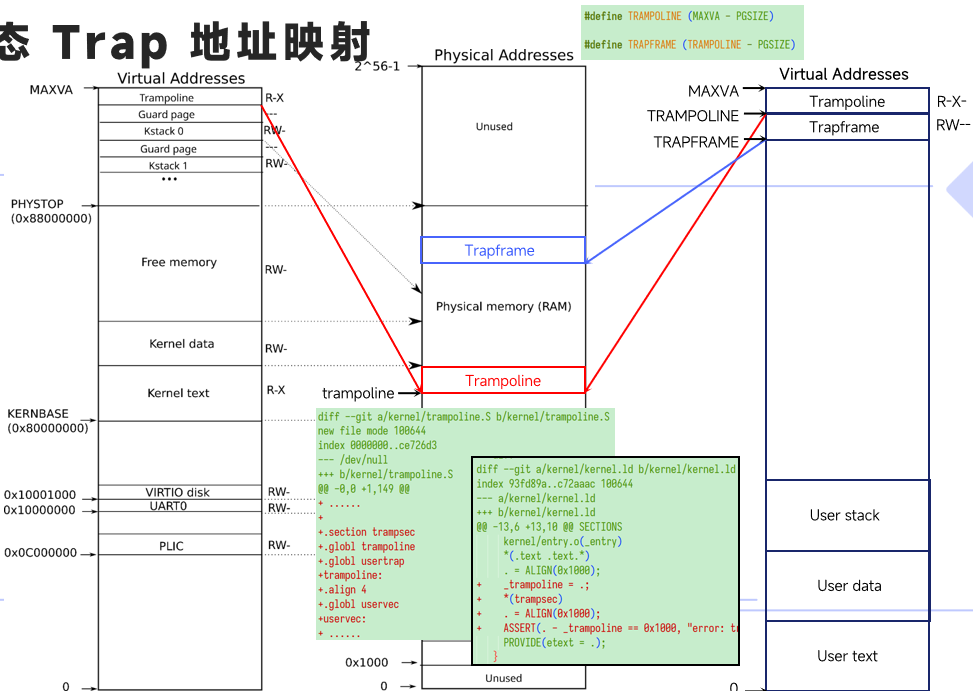
frame);

0

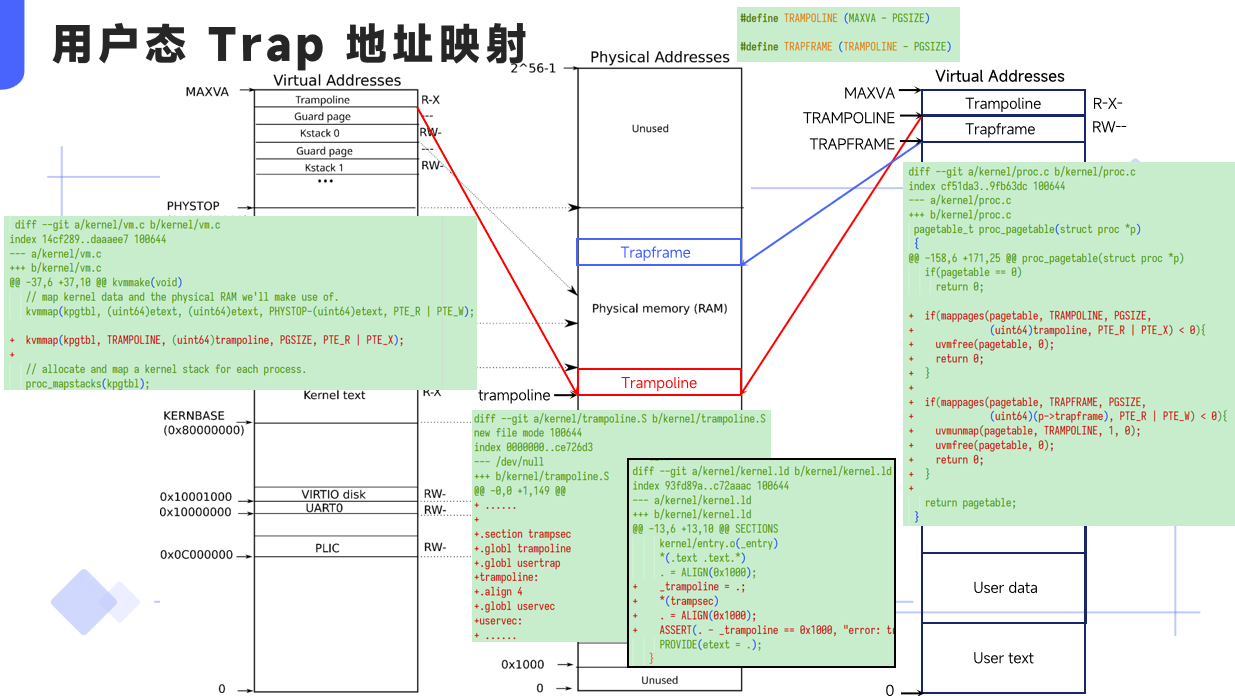
用户态 Trap 地址映射



用户态 Trap 地址映射



用户态 Trap 地址映射



用户态 Trap 的处理流程

kernel/trampoline.S

```
uservec:
    csrw sscratch, a0
    li a0, TRAPFRAME
    sd ra, 40(a0)
    .....
    sd t6, 280(a0)
    csrr t0, sscratch
    sd t0, 112(a0)

    ld sp, 8(a0)
    ld tp, 32(a0)
    ld t0, 16(a0)
    ld t1, 0(a0)

    sfence.vma zero, zero
    csrw satp, t1
    sfence.vma zero, zero

    jalr t0

userret:
    sfence.vma zero, zero
    csrw satp, a0
    sfence.vma zero, zero

    li a0, TRAPFRAME
    ld ra, 40(a0)
    .....
    ld t6, 280(a0)
    ld a0, 112(a0)

    sret
```

用户态 Trap 的汇编

kernel/trampoline.S

uservec:

```
csrw sscratch, a0
li a0, TRAPFRAME
sd ra, 40(a0)
.....
sd t6, 280(a0)
csrr t0, sscratch
sd t0, 112(a0)
```

```
ld sp, 8(a0)
ld tp, 32(a0)
ld t0, 16(a0)
ld t1, 0(a0)
```

```
sfence.vma zero, zero
csrw satp, t1
sfence.vma zero, zero
```

```
jalr t0
```

userret:

```
sfence.vma zero, zero
csrw satp, a0
sfence.vma zero, zero
```

```
li a0, TRAPFRAME
ld ra, 40(a0)
.....
ld t6, 280(a0)
ld a0, 112(a0)
```

```
sret
```

```
sscratch = a0;
a0 = (struct trapframe *)&trapframe;
a0->ra = ra;
...
a0->t6 = t6;
t0 = sscratch;
a0->a0 = t0;
```

用户态 Trap 的

kernel/trampoline.S

uservec:

```
csrw sscratch, a0
li a0, TRAPFRAME
sd ra, 40(a0)
.....
sd t6, 280(a0)
csrr t0, sscratch
sd t0, 112(a0)
```

```
ld sp, 8(a0)
ld tp, 32(a0)
ld t0, 16(a0)
ld t1, 0(a0)
```

```
sfence.vma zero, zero
csrw satp, t1
sfence.vma zero, zero
```

```
jalr t0
```

userret:

```
sfence.vma zero, zero
csrw satp, a0
sfence.vma zero, zero
```

```
li a0, TRAPFRAME
ld ra, 40(a0)
.....
ld t6, 280(a0)
ld a0, 112(a0)
```

```
sret
```

```
sscratch = a0;
a0 = (struct trapframe *)&trapframe;
a0->ra = ra;
...
a0->t6 = t6;
t0 = sscratch;
a0->a0 = t0;
```

```
struct trapframe {
    /* 0 */ uint64 kernel_satp; // kernel page table
    /* 8 */ uint64 kernel_sp; // top of process's kernel stack
    /* 16 */ uint64 kernel_trap; // usertrap()
    /* 24 */ uint64 epc; // saved user program counter
    /* 32 */ uint64 kernel_hartid; // saved kernel tp
```

sp = a0->kernel_sp; // 恢复内核栈指针
tp = a0->kernel_hartid; // 用保存的当前 hartid 恢复 tp
t0 = a0->kernel_trap; // 用 t0 保存 usertrap() 的地址，以备后用
t1 = a0->kernel_satp; // 用 t1 保存内核页表的地址，以备后用

用户态 Trap 的

kernel/trampoline.S

```
uservec:
    csrw sscratch, a0
    li a0, TRAPFRAME
    sd ra, 40(a0)
    .....
    sd t6, 280(a0)
    csrr t0, sscratch
    sd t0, 112(a0)

    ld sp, 8(a0)
    ld tp, 32(a0)
    ld t0, 16(a0)
    ld t1, 0(a0)

    sfence.vma zero, zero
    csrw satp, t1
    sfence.vma zero, zero

    jalr t0

userret:
    sfence.vma zero, zero
    csrw satp, a0
    sfence.vma zero, zero

    li a0, TRAPFRAME
    ld ra, 40(a0)
    .....
    ld t6, 280(a0)
    ld a0, 112(a0)

    sret
```

```
sscratch = a0;
a0 = (struct trapframe *)&trapframe;
a0->ra = ra;
...
a0->t6 = t6;
t0 = sscratch;
a0->a0 = t0;
```

```
struct trapframe {
    /* 0 */ uint64 kernel_satp; // kernel page table
    /* 8 */ uint64 kernel_sp; // top of process's kernel stack
    /* 16 */ uint64 kernel_trap; // usertrap()
    /* 24 */ uint64 epc; // saved user program counter
    /* 32 */ uint64 kernel_hartid; // saved kernel tp
```

sp = a0->kernel_sp; // 恢复内核栈指针
tp = a0->kernel_hartid; // 用保存的当前 hartid 恢复 tp
t0 = a0->kernel_trap; // 用 t0 保存 usertrap() 的地址, 以备后用
t1 = a0->kernel_satp; // 用 t1 保存内核页表的地址, 以备后用

将 satp 设置为 t1, 即替换为内核页表地址

用户态 Trap 的汇编

kernel/trampoline.S

uservec:

```
csrw sscratch, a0
li a0, TRAPFRAME
sd ra, 40(a0)
.....
sd t6, 280(a0)
csrr t0, sscratch
sd t0, 112(a0)
```

```
ld sp, 8(a0)
ld tp, 32(a0)
ld t0, 16(a0)
ld t1, 0(a0)
```

```
sfence.vma zero, zero
csrw satp, t1
sfence.vma zero, zero
```

jalr t0

userret:

```
sfence.vma zero, zero
csrw satp, a0
sfence.vma zero, zero
```

```
li a0, TRAPFRAME
ld ra, 40(a0)
.....
ld t6, 280(a0)
ld a0, 112(a0)
```

sret

```
sscratch = a0;
a0 = (struct trapframe *)&trapframe;
a0->ra = ra;
...
a0->t6 = t6;
t0 = sscratch;
a0->a0 = t0;
```

```
struct trapframe {
    /* 0 */ uint64 kernel_satp; // kernel page table
    /* 8 */ uint64 kernel_sp; // top of process's kernel stack
    /* 16 */ uint64 kernel_trap; // usertrap()
    /* 24 */ uint64 epc; // saved user program counter
    /* 32 */ uint64 kernel_hartid; // saved kernel tp
```

sp = a0->kernel_sp; // 恢复内核栈指针
tp = a0->kernel_hartid; // 用保存的当前 hartid 恢复 tp
t0 = a0->kernel_trap; // 用 t0 保存 usertrap() 的地址, 以备后用
t1 = a0->kernel_satp; // 用 t1 保存内核页表的地址, 以备后用

将 satp 设置为 t1, 即替换为内核页表地址

展开为 jalr ra, t0, 0

- 跳转到 t0 寄存器中存储的地址 (usertrap())
- PC+4 作为返回地址保存在 ra 寄存器中

用户态 Trap 的处理流程

kernel/trampoline.S

```
uservec:
    csrw sscratch, a0
    li a0, TRAPFRAME
    sd ra, 40(a0)
    .....
    sd t6, 280(a0)
    csrr t0, sscratch
    sd t0, 112(a0)

    ld sp, 8(a0)
    ld tp, 32(a0)
    ld t0, 16(a0)
    ld t1, 0(a0)

    sfence.vma zero, zero
    csrw satp, t1
    sfence.vma zero, zero

    jalr t0

userret:
    sfence.vma zero, zero
    csrw satp, a0
    sfence.vma zero, zero

    li a0, TRAPFRAME
    ld ra, 40(a0)
    .....
    ld t6, 280(a0)
    ld a0, 112(a0)

    sret
```

kernel/trap.c

```
uint64 usertrap(void)
{
    int which_dev = 0;

    if((r_sstatus() & SSTATUS_SPP) != 0)
        panic("usertrap: not from user mode");

    w_stvec((uint64)kernelvec); //DOC: kernelvec

    struct proc *p = myproc();

    p->trapframe->epc = r_sepc();

    if(r_scause() == 8){
        panic("usertrap: syscall not implemented yet");
    } else if((which_dev = devintr()) != 0){
        // ok
    } else {
        printf("usertrap(): unexpected scause 0x%x pid=%d\n", r_scause(), p->pid);
        printf("          sepc=0x%x stval=0x%x\n", r_sepc(), r_stval());
        panic("usertrap: should be killed but we have not support killing!");
    }

    if(which_dev == 2)
        yield();

    prepare_return();

    uint64 satp = MAKE_SATP(p->pagetable);

    return satp;
}
```

用户态 Trap 的处理流程

kernel/trampoline.S

```
uservec:
    csrw sscratch, a0
    li a0, TRAPFRAME
    sd ra, 40(a0)
    .....
    sd t6, 280(a0)
    csrr t0, sscratch
    sd t0, 112(a0)

    ld sp, 8(a0)
    ld tp, 32(a0)
    ld t0, 16(a0)
    ld t1, 0(a0)

    sfence.vma zero, zero
    csrw satp, t1
    sfence.vma zero, zero

    jalr t0

userret:
    sfence.vma zero, zero
    csrw satp, a0
    sfence.vma zero, zero

    li a0, TRAPFRAME
    ld ra, 40(a0)
    .....
    ld t6, 280(a0)
    ld a0, 112(a0)

    sret
```

kernel/trap.c

```
uint64 usertrap(void)
{
    int which_dev = 0;

    if((r_sstatus() & SSTATUS_SPP) != 0)
        panic("usertrap: not from user mode");

    w_stvec((uint64)kernelvec); //DOC: kernelvec

    struct proc *p = myproc();

    p->trapframe->epc = r_sepc();

    if(r_scause() == 8){
        panic("usertrap: syscall not implemented yet");
    } else if((which_dev = devintr()) != 0){
        // ok
    } else {
        printf("usertrap(): unexpected scause 0x%x pid=%d\n", r_scause(), p->pid);
        printf("          sepc=0x%x stval=0x%x\n", r_sepc(), r_stval());
        panic("usertrap: should be killed but we have not support killing!");
    }

    if(which_dev == 2)
        yield();

    prepare_return();

    uint64 satp = MAKE_SATP(p->pagetable);

    return satp;
}
```

用户态 Trap 的处理流程

kernel/trampoline.S

```
uservec:
    csrw sscratch, a0
    li a0, TRAPFRAME
    sd ra, 40(a0)
    .....
    sd t6, 280(a0)
    csrr t0, sscratch
    sd t0, 112(a0)

    ld sp, 8(a0)
    ld tp, 32(a0)
    ld t0, 16(a0)
    ld t1, 0(a0)

    sfence.vma zero, zero
    csrw satp, t1
    sfence.vma zero, zero

    jalr t0
```

```
userret:
    sfence.vma zero, zero
    csrw satp, a0
    sfence.vma zero, zero

    li a0, TRAPFRAME
    ld ra, 40(a0)
    .....
    ld t6, 280(a0)
    ld a0, 112(a0)

    sret
```

kernel/trap.c

```
uint64 usertrap(void)
{
    int which_dev = 0;

    if((r_sstatus() & SSTATUS_SPP) != 0)
        panic("usertrap: not from user mode");

    w_stvec((uint64)kernelvec); //DOC: kernelvec

    struct proc *p = myproc();

    p->trapframe->epc = r_sepc();

    if(r_scause() == 8){
        panic("usertrap: syscall not implemented yet");
    } else if((which_dev = devintr()) != 0){
        // ok
    } else {
        printf("usertrap(): unexpected scause 0x%x pid=%d\n", r_scause(), p->pid);
        printf("          sepc=0x%x stval=0x%x\n", r_sepc(), r_stval());
        panic("usertrap: should be killed but we have not support killing!");
    }

    if(which_dev == 2)
        yield();

    prepare_return();

    uint64 satp = MAKE_SATP(p->pagetable);

    return satp;
}
```


用户态 Trap 的处理流程

kernel/trampoline.S

```
uservec:
    csrw sscratch, a0
    li a0, TRAPFRAME
    sd ra, 40(a0)
    .....
    sd t6, 280(a0)
    csrr t0, sscratch
    sd t0, 112(a0)

    ld sp, 8(a0)
    ld tp, 32(a0)
    ld t0, 16(a0)
    ld t1, 0(a0)

    sfence.vma zero, zero
    csrw satp, t1
    sfence.vma zero, zero

    jalr t0

userret:
    sfence.vma zero, zero
    csrw satp, a0
    sfence.vma zero, zero

    li a0, TRAPFRAME
    ld ra, 40(a0)
    .....
    ld t6, 280(a0)
    ld a0, 112(a0)

    sret
```

kernel/trap.c

```
uint64 usertrap(void)
{
    int which_dev = 0;

    if((r_sstatus() & SSTATUS_SPP) != 0)
        panic("usertrap: not from user mode");

    w_stvec((uint64)kernelvec); //DOC: kernelvec

    struct proc *p = myproc();

    p->trapframe->epc = r_sepc();

    if(r_scause() == 8){
        panic("usertrap: syscall not implemented yet");
    } else if((which_dev = devintr()) != 0){
        // ok
    } else {
        printf("usertrap(): unexpected scause 0x%x pid=%d\n", r_scause(), p->pid);
        printf("          sepc=0x%x stval=0x%x\n", r_sepc(), r_stval());
        panic("usertrap: should be killed but we have not support killing!");
    }

    if(which_dev == 2)
        yield();

    prepare_return();

    uint64 satp = MAKE_SATP(p->pagetable);

    return satp;
}
```

用户态 Trap 的处理流程

kernel/trampoline.S

```
uservec:
    csrw sscratch, a0
    li a0, TRAPFRAME
    sd ra, 40(a0)
    .....
    sd t6, 280(a0)
    csrr t0, sscratch
    sd t0, 112(a0)

    ld sp, 8(a0)
    ld tp, 32(a0)
    ld t0, 16(a0)
    ld t1, 0(a0)

    sfence.vma zero, zero
    csrw satp, t1
    sfence.vma zero, zero

    jalr t0
```

```
userret:
    sfence.vma zero, zero
    csrw satp, a0
    sfence.vma zero, zero

    li a0, TRAPFRAME
    ld ra, 40(a0)
    .....
    ld t6, 280(a0)
    ld a0, 112(a0)

    sret
```

kernel/trap.c

```
uint64 usertrap(void)
{
    int which_dev = 0;

    if((r_sstatus() & SSTATUS_SPP) != 0)
        panic("usertrap: not from user mode");

    w_stvec((uint64)kernelvec); //DOC: kernelvec

    struct proc *p = myproc();

    p->trapframe->epc = r_sepc();

    if(r_scause() == 8){
        panic("usertrap: syscall not implemented yet");
    } else if((which_dev = devintr()) != 0){
        // ok
    } else {
        printf("usertrap(): unexpected scause 0x%x pid=%d\n", r_scause(), p->pid);
        printf("          sepc=0x%x stval=0x%x\n", r_sepc(), r_stval());
        panic("usertrap: should be killed but we have not support killing!");
    }

    if(which_dev == 2)
        yield();

    prepare_return();

    uint64 satp = MAKE_SATP(p->pagetable);

    return satp;
}
```

用户态 Trap 的处理流程

kernel/trampoline.S

```
uservec:
    csrw sscratch, a0
    li a0, TRAPFRAME
    sd ra, 40(a0)
    .....
    sd t6, 280(a0)
    csrr t0, sscratch
    sd t0, 112(a0)

    ld sp, 8(a0)
    ld tp, 32(a0)
    ld t0, 16(a0)
    ld t1, 0(a0)

    sfence.vma zero, zero
    csrw satp, t1
    sfence.vma zero, zero

    jalr t0

userret:
    sfence.vma zero, zero
    csrw satp, a0
    sfence.vma zero, zero

    li a0, TRAPFRAME
    ld ra, 40(a0)
    .....
    ld t6, 280(a0)
    ld a0, 112(a0)

    sret
```

kernel/trap.c

```
uint64 usertrap(void)
{
    int which_dev = 0;

    if((r_sstatus() & SSTATUS_SPP) != 0)
        panic("usertrap: not from user mode");

    w_stvec((uint64)kernelvec); //DOC: kernelvec

    struct proc *p = myproc();

    p->trapframe->epc = r_sepc();

    if(r_scause() == 8){
        panic("usertrap: syscall not implemented");
    } else if((which_dev = devintr()) != 0){
        // ok
    } else {
        printf("usertrap(): unexpected scause\n");
        printf("             sepc=0x%lx stval=0x%lx\n", r_sepc(), r_stval());
        panic("usertrap: should be killed but here");
    }

    if(which_dev == 2)
        yield();

    prepare_return();

    uint64 satp = MAKE_SATP(p->pagetable);

    return satp;
}
```

```
void prepare_return(void)
{
    struct proc *p = myproc();

    intr_off();

    uint64 trampoline_uservec = TRAPLINE + (uservec - trampoline);
    w_stvec(trampoline_uservec);

    p->trapframe->kernel_satp = r_satp(); // kernel page table
    p->trapframe->kernel_sp = p->kstack + PGSIZE; // process's kernel stack
    p->trapframe->kernel_trap = (uint64)usertrap;
    p->trapframe->kernel_hartid = r_tp(); // hartid for cpuid()

    unsigned long x = r_sstatus();
    x &= ~SSTATUS_SPP; // clear SPP to 0 for user mode
    x |= SSTATUS_SPIE; // enable interrupts in user mode
    w_sstatus(x);

    w_sepc(p->trapframe->epc);
}
```

用户态 Trap 的处理流程

kernel/trampoline.S

```
uservec:
    csrw sscratch, a0
    li a0, TRAPFRAME
    sd ra, 40(a0)
    .....
    sd t6, 280(a0)
    csrr t0, sscratch
    sd t0, 112(a0)

    ld sp, 8(a0)
    ld tp, 32(a0)
    ld t0, 16(a0)
    ld t1, 0(a0)

    sfence.vma zero, zero
    csrw satp, t1
    sfence.vma zero, zero

    jalr t0

userret:
    sfence.vma zero, zero
    csrw satp, a0
    sfence.vma zero, zero

    li a0, TRAPFRAME
    ld ra, 40(a0)
    .....
    ld t6, 280(a0)
    ld a0, 112(a0)

    sret
```

kernel/trap.c

```
uint64 usertrap(void)
{
    int which_dev = 0;

    if((r_sstatus() & SSTATUS_SPP) != 0)
        panic("usertrap: not from user mode");

    w_stvec((uint64)kernelvec); //DOC: kernelvec

    struct proc *p = myproc();

    p->trapframe->epc = r_sepc();

    if(r_scause() == 8){
        panic("usertrap: syscall not implemented");
    } else if((which_dev = devintr()) != 0){
        // ok
    } else {
        printf("usertrap(): unexpected scause\n");
        printf("             sepc=0x%16lx\n", r_sepc());
        panic("usertrap: should be killed by kernel");
    }

    if(which_dev == 2)
        yield();

    prepare_return();

    uint64 satp = MAKE_SATP(p->pagetable);

    return satp;
}
```

kernel/trampoline.S

```
.section trampsec
.globl trampoline
.globl usertrap
trampoline:
.align 4
.globl uservec
uservec:
```

void prepare_return(void)

```
{
    struct proc *p = myproc();
    intr_off();
}
```

```
uint64 trampoline_uservec = TRAMPOLINE + (uservec - trampoline);
w_stvec(trampoline_uservec);
```

```
p->trapframe->kernel_satp = r_satp(); // kernel page table
p->trapframe->kernel_sp = p->kstack + PGSIZE; // process's kernel stack
p->trapframe->kernel_trap = (uint64)usertrap;
p->trapframe->kernel_hartid = r_tp(); // hartid for cpuid()
```

```
unsigned long x = r_sstatus();
x &= ~SSTATUS_SPP; // clear SPP to 0 for user mode
x |= SSTATUS_SPIE; // enable interrupts in user mode
w_sstatus(x);
```

```
w_sepc(p->trapframe->epc);
}
```

用户态 Trap 的处理流程

kernel/trampoline.S

```
uservec:
    csrw sscratch, a0
    li a0, TRAPFRAME
    sd ra, 40(a0)
    .....
    sd t6, 280(a0)
    csrr t0, sscratch
    sd t0, 112(a0)

    ld sp, 8(a0)
    ld tp, 32(a0)
    ld t0, 16(a0)
    ld t1, 0(a0)

    sfence.vma zero, zero
    csrw satp, t1
    sfence.vma zero, zero

    jalr t0

userret:
    sfence.vma zero, zero
    csrw satp, a0
    sfence.vma zero, zero

    li a0, TRAPFRAME
    ld ra, 40(a0)
    .....
    ld t6, 280(a0)
    ld a0, 112(a0)

    sret
```

kernel/trap.c

```
uint64 usertrap(void)
{
    int which_dev = 0;

    if((r_sstatus() & SSTATUS_SPP) != 0)
        panic("usertrap: not from user mode");

    w_stvec((uint64)kernelvec); //DOC: kernelvec

    struct proc *p = myproc();

    p->trapframe->epc = r_sepc();

    if(r_scause() == 8){
        panic("usertrap: syscall not implemented");
    } else if((which_dev = devintr()) != 0){
        // ok
    } else {
        printf("usertrap(): unexpected scause\n");
        printf("             sepc=0x%lx stval=0x%lx\n", r_sepc(), r_stval());
        panic("usertrap: should be killed but here");
    }

    if(which_dev == 2)
        yield();

    prepare_return();

    uint64 satp = MAKE_SATP(p->pagetable);

    return satp;
}
```

```
void prepare_return(void)
{
    struct proc *p = myproc();

    intr_off();

    uint64 trampoline_uservec = TRAMPOLINE + (uservec - trampoline);
    w_stvec(trampoline_uservec);

    p->trapframe->kernel_satp = r_satp(); // kernel page table
    p->trapframe->kernel_sp = p->kstack + PGSIZE; // process's kernel stack
    p->trapframe->kernel_trap = (uint64)usertrap;
    p->trapframe->kernel_hartid = r_tp(); // hartid for cpuid()

    unsigned long x = r_sstatus();
    x &= ~SSTATUS_SPP; // clear SPP to 0 for user mode
    x |= SSTATUS_SPIE; // enable interrupts in user mode
    w_sstatus(x);

    w_sepc(p->trapframe->epc);
}
```

用户态 Trap 的处理流程

kernel/trampoline.S

```
uservec:
    csrw sscratch, a0
    li a0, TRAPFRAME
    sd ra, 40(a0)
    .....
    sd t6, 280(a0)
    csrr t0, sscratch
    sd t0, 112(a0)

    ld sp, 8(a0)
    ld tp, 32(a0)
    ld t0, 16(a0)
    ld t1, 0(a0)

    sfence.vma zero, zero
    csrw satp, t1
    sfence.vma zero, zero

    jalr t0

userret:
    sfence.vma zero, zero
    csrw satp, a0
    sfence.vma zero, zero

    li a0, TRAPFRAME
    ld ra, 40(a0)
    .....
    ld t6, 280(a0)
    ld a0, 112(a0)

    sret
```

kernel/trap.c

```
uint64 usertrap(void)
{
    int which_dev = 0;

    if((r_sstatus() & SSTATUS_SPP) != 0)
        panic("usertrap: not from user mode");

    w_stvec((uint64)kernelvec); //DOC: kernelvec

    struct proc *p = myproc();

    p->trapframe->epc = r_sepc();

    if(r_scause() == 8){
        panic("usertrap: syscall not implemented");
    } else if((which_dev = devintr()) != 0){
        // ok
    } else {
        printf("usertrap(): unexpected scause\n");
        printf("             sepc=0x%lx stval=%lx\n", r_sepc(), r_stval());
        panic("usertrap: should be killed but not");
    }

    if(which_dev == 2)
        yield();

    prepare_return();

    uint64 satp = MAKE_SATP(p->pagetable);

    return satp;
}
```

```
void prepare_return(void)
{
    struct proc *p = myproc();

    intr_off();

    uint64 trampoline_uservec = TRAMPOLINE + (uservec - trampoline);
    w_stvec(trampoline_uservec);

    p->trapframe->kernel_satp = r_satp(); // kernel page table
    p->trapframe->kernel_sp = p->kstack + PGSIZE; // process's kernel stack
    p->trapframe->kernel_trap = (uint64)usertrap;
    p->trapframe->kernel_hartid = r_tp(); // hartid for cpuid()

    unsigned long x = r_sstatus();
    x &= ~SSTATUS_SPP; // clear SPP to 0 for user mode
    x |= SSTATUS_SPIE; // enable interrupts in user mode
    w_sstatus(x);

    w_sepc(p->trapframe->epc);
}
```

用户态 Trap 的处理流程

kernel/trampoline.S

```
uservec:
    csrw sscratch, a0
    li a0, TRAPFRAME
    sd ra, 40(a0)
    .....
    sd t6, 280(a0)
    csrr t0, sscratch
    sd t0, 112(a0)

    ld sp, 8(a0)
    ld tp, 32(a0)
    ld t0, 16(a0)
    ld t1, 0(a0)

    sfence.vma zero, zero
    csrw satp, t1
    sfence.vma zero, zero

    jalr t0

userret:
    sfence.vma zero, zero
    csrw satp, a0
    sfence.vma zero, zero

    li a0, TRAPFRAME
    ld ra, 40(a0)
    .....
    ld t6, 280(a0)
    ld a0, 112(a0)

    sret
```

kernel/trap.c

```
uint64 usertrap(void)
{
    int which_dev = 0;

    if((r_sstatus() & SSTATUS_SPP) != 0)
        panic("usertrap: not from user mode");

    w_stvec((uint64)kernelvec); //DOC: kernelvec

    struct proc *p = myproc();

    p->trapframe->epc = r_sepc();

    if(r_scause() == 8){
        panic("usertrap: syscall not implemented");
    } else if((which_dev = devintr()) != 0){
        // ok
    } else {
        printf("usertrap(): unexpected scause\n");
        printf("             sepc=0x%lx stval=0x%lx\n", r_sepc(), r_stval());
        panic("usertrap: should be killed but here");
    }

    if(which_dev == 2)
        yield();

    prepare_return();

    uint64 satp = MAKE_SATP(p->pagetable);

    return satp;
}
```

```
void prepare_return(void)
{
    struct proc *p = myproc();

    intr_off();

    uint64 trampoline_uservec = TRAMPOLINE + (uservec - trampoline);
    w_stvec(trampoline_uservec);

    p->trapframe->kernel_satp = r_satp(); // kernel page table
    p->trapframe->kernel_sp = p->kstack + PGSIZE; // process's kernel stack
    p->trapframe->kernel_trap = (uint64)usertrap;
    p->trapframe->kernel_hartid = r_tp(); // hartid for cpuid()

    unsigned long x = r_sstatus();
    x &= ~SSTATUS_SPP; // clear SPP to 0 for user mode
    x |= SSTATUS_SPIE; // enable interrupts in user mode
    w_sstatus(x);

    w_sepc(p->trapframe->epc);
}
```

用户态 Trap 的处理流程

kernel/trampoline.S

```
uservec:
    csrw sscratch, a0
    li a0, TRAPFRAME
    sd ra, 40(a0)
    .....
    sd t6, 280(a0)
    csrr t0, sscratch
    sd t0, 112(a0)

    ld sp, 8(a0)
    ld tp, 32(a0)
    ld t0, 16(a0)
    ld t1, 0(a0)

    sfence.vma zero, zero
    csrw satp, t1
    sfence.vma zero, zero

    jalr t0

userret:
    sfence.vma zero, zero
    csrw satp, a0
    sfence.vma zero, zero

    li a0, TRAPFRAME
    ld ra, 40(a0)
    .....
    ld t6, 280(a0)
    ld a0, 112(a0)

    sret
```

kernel/trap.c

```
uint64 usertrap(void)
{
    int which_dev = 0;

    if((r_sstatus() & SSTATUS_SPP) != 0)
        panic("usertrap: not from user mode");

    w_stvec((uint64)kernelvec); //DOC: kernelvec

    struct proc *p = myproc();

    p->trapframe->epc = r_sepc();

    if(r_scause() == 8){
        panic("usertrap: syscall not implemented yet");
    } else if((which_dev = devintr()) != 0){
        // ok
    } else {
        printf("usertrap(): unexpected scause 0x%x pid=%d\n", r_scause(), p->pid);
        printf("          sepc=0x%x stval=0x%x\n", r_sepc(), r_stval());
        panic("usertrap: should be killed but we have not support killing!");
    }

    if(which_dev == 2)
        yield();

    prepare_return();

    uint64 satp = MAKE_SATP(p->pagetable);

    return satp;
}
```


用户态 Trap 的处理流程

kernel/trampoline.S

```
uservec:
    csrw sscratch, a0
    li a0, TRAPFRAME
    sd ra, 40(a0)
    .....
    sd t6, 280(a0)
    csrr t0, sscratch
    sd t0, 112(a0)

    ld sp, 8(a0)
    ld tp, 32(a0)
    ld t0, 16(a0)
    ld t1, 0(a0)

    sfence.vma zero, zero
    csrw satp, t1
    sfence.vma zero, zero

    jalr t0

userret:
    sfence.vma zero, zero
    csrw satp, a0
    sfence.vma zero, zero

    li a0, TRAPFRAME
    ld ra, 40(a0)
    .....
    ld t6, 280(a0)
    ld a0, 112(a0)

    sret
```

kernel/trap.c

```
uint64 usertrap(void)
{
    int which_dev = 0;

    if((r_sstatus() & SSTATUS_SPP) != 0)
        panic("usertrap: not from user mode");

    w_stvec((uint64)kernelvec); //DOC: kernelvec

    struct proc *p = myproc();

    p->trapframe->epc = r_sepc();

    if(r_scause() == 8){
        panic("usertrap: syscall not implemented yet");
    } else if((which_dev = devintr()) != 0){
        // ok
    } else {
        printf("usertrap(): unexpected scause 0x%x pid=%d\n", r_scause(), p->pid);
        printf("          sepc=0x%x stval=0x%x\n", r_sepc(), r_stval());
        panic("usertrap: should be killed but we have not support killing!");
    }

    if(which_dev == 2)
        yield();

    prepare_return();

    uint64 satp = MAKE_SATP(p->pagetable);

    return satp;
}
```

用户态 Trap 的处理流程

kernel/trampoline.S

```
uservec:
    csrw sscratch, a0
    li a0, TRAPFRAME
    sd ra, 40(a0)
    .....
    sd t6, 280(a0)
    csrr t0, sscratch
    sd t0, 112(a0)

    ld sp, 8(a0)
    ld tp, 32(a0)
    ld t0, 16(a0)
    ld t1, 0(a0)

    sfence.vma zero, zero
    csrw satp, t1
    sfence.vma zero, zero

    jalr t0

userret:
    sfence.vma zero, zero
    csrw satp, a0
    sfence.vma zero, zero

    li a0, TRAPFRAME
    ld ra, 40(a0)
    .....
    ld t6, 280(a0)
    ld a0, 112(a0)

    sret
```

kernel/trap.c

```
uint64 usertrap(void)
{
    int which_dev = 0;

    if((r_sstatus() & SSTATUS_SPP) != 0)
        panic("usertrap: not from user mode");

    w_stvec((uint64)kernelvec); //DOC: kernelvec

    struct proc *p = myproc();

    p->trapframe->epc = r_sepc();

    if(r_scause() == 8){
        panic("usertrap: syscall not implemented yet");
    } else if((which_dev = devintr()) != 0){
        // ok
    } else {
        cause(), p->pid);
        stval();
        killing!");
    }

    if(which_dev == 2)
        yield();

    prepare_return();

    uint64 satp = MAKE_SATP(r_sepc());

    return satp;
}
```

展开为 jalr ra, t0, 0

- 跳转到 t0 寄存器中存储的地址 (usertrap())
- PC+4 作为返回地址保存在 ra 寄存器中

展开为:

a0 = satp
jalr x0, ra, 0

等价于 C 的: userret(satp);

用户态 Trap 的处理流程

kernel/trampoline.S

```
uservec:
    csrw sscratch, a0
    li a0, TRAPFRAME
    sd ra, 40(a0)
    .....
    sd t6, 280(a0)
    csrr t0, sscratch
    sd t0, 112(a0)

    ld sp, 8(a0)
    ld tp, 32(a0)
    ld t0, 16(a0)
    ld t1, 0(a0)

    sfence.vma zero, zero
    csrw satp, t1
    sfence.vma zero, zero

    jalr t0

userret:
    sfence.vma zero, zero
    csrw satp, a0
    sfence.vma zero, zero

    li a0, TRAPFRAME
    ld ra, 40(a0)
    .....
    ld t6, 280(a0)
    ld a0, 112(a0)

    sret
```

kernel/trap.c

```
uint64 usertrap(void)
{
    int which_dev = 0;

    if((r_sstatus() & SSTATUS_SPP) != 0)
        panic("usertrap: not from user mode");

    w_stvec((uint64)kernelvec); //DOC: kernelvec

    struct proc *p = myproc();

    p->trapframe->epc = r_sepc();

    if(r_scause() == 8){
        panic("usertrap: syscall not implemented yet");
    } else if((which_dev = devintr()) != 0){
        // ok
    } else {
        printf("usertrap(): unexpected scause 0x%x pid=%d\n", r_scause(), p->pid);
        printf("          sepc=0x%x stval=0x%x\n", r_sepc(), r_stval());
        panic("usertrap: should be killed but we have not support killing!");
    }

    if(which_dev == 2)
        yield();

    prepare_return();

    uint64 atp = MAKE_SATP(p->pagetable);

    return satp;
}
```

用户态 Trap 的处理流程

kernel/trampoline.S

```
uservec:
    csrw sscratch, a0
    li a0, TRAPFRAME
    sd ra, 40(a0)
    .....
    sd t6, 280(a0)
    csrr t0, sscratch
    sd t0, 112(a0)

    ld sp, 8(a0)
    ld tp, 32(a0)
    ld t0, 16(a0)
    ld t1, 0(a0)

    sfence.vma zero, zero
    csrw satp, t1
    sfence.vma zero, zero

    jalr t0

userret:
    sfence.vma zero, zero
    csrw satp, a0
    sfence.vma zero, zero

    li a0, TRAPFRAME
    ld ra, 40(a0)
    .....
    ld t6, 280(a0)
    ld a0, 112(a0)

    sret
```

kernel/trap.c

```
uint64 usertrap(void)
{
    int which_dev = 0;

    if((r_sstatus() & SSTATUS_SPP) != 0)
        panic("usertrap: not from user mode");

    w_stvec((uint64)kernelvec); //DOC: kernelvec

    struct proc *p = myproc();

    p->trapframe->epc = r_sepc();

    if(r_scause() == 8){
        panic("usertrap: syscall not implemented yet");
    } else if((which_dev = devintr()) != 0){
        // ok
    } else {
        printf("usertrap(): unexpected scause 0x%x pid=%d\n", r_scause(), p->pid);
        printf("          sepc=0x%x stval=0x%x\n", r_sepc(), r_stval());
        panic("usertrap: should be killed but we have not support killing!");
    }

    if(which_dev == 2)
        yield();

    prepare_return();

    uint64 atp = MAKE_SATP(p->pagetable);

    return satp;
}
```

用户态 Trap 的处理流程

kernel/trampoline.S

```
uservec:
    csrw sscratch, a0
    li a0, TRAPFRAME
    sd ra, 40(a0)
    .....
    sd t6, 280(a0)
    csrr t0, sscratch
    sd t0, 112(a0)

    ld sp, 8(a0)
    ld tp, 32(a0)
    ld t0, 16(a0)
    ld t1, 0(a0)

    sfence.vma zero, zero
    csrw satp, t1
    sfence.vma zero, zero

    jalr t0

userret:
    sfence.vma zero, zero
    csrw satp, a0
    sfence.vma zero, zero

    li a0, TRAPFRAME
    ld ra, 40(a0)
    .....
    ld t6, 280(a0)
    ld a0, 112(a0)

    sret
```

kernel/trap.c

```
uint64 usertrap(void)
{
    int which_dev = 0;

    if((r_sstatus() & SSTATUS_SPP) != 0)
        panic("usertrap: not from user mode");

    w_stvec((uint64)kernelvec); //DOC: kernelvec

    struct proc *p = myproc();

    p->trapframe->epc = r_sepc();

    if(r_scause() == 8){
        panic("usertrap: syscall not implemented yet");
    } else if((which_dev = devintr()) != 0){
        // ok
    } else {
        printf("usertrap(): unexpected scause 0x%x pid=%d\n", r_scause(), p->pid);
        printf("          sepc=0x%x stval=0x%x\n", r_sepc(), r_stval());
        panic("usertrap: should be killed but we have not support killing!");
    }

    if(which_dev == 2)
        yield();

    prepare_return();

    uint64 atp = MAKE_SATP(p->pagetable);

    return satp;
}
```

用户态 Trap 的处理流程

kernel/trampoline.S

```
uservec:
    csrw sscratch, a0
    li a0, TRAPFRAME
    sd ra, 40(a0)
    .....
    sd t6, 280(a0)
    csrr t0, sscratch
    sd t0, 112(a0)

    ld sp, 8(a0)
    ld tp, 32(a0)
    ld t0, 16(a0)
    ld t1, 0(a0)

    sfence.vma zero, zero
    csrw satp, t1
    sfence.vma zero, zero

    jalr t0

userret:
    sfence.vma zero, zero
    csrw satp, a0
    sfence.vma zero, zero

    li a0, TRAPFRAME
    ld ra, 40(a0)
    .....
    ld t6, 280(a0)
    ld a0, 112(a0)
```

sret

kernel/trap.c

```
uint64 usertrap(void)
{
    int which_dev = 0;

    if((r_sstatus() & SSTATUS_SPP) != 0)
        panic("usertrap: not from user mode");

    w_stvec((uint64)0x00000000);

    struct proc *p =
        MAXVA
    p->trapframe->epc =
    TRAPFRAME
    if(_sstatus()
        panic("usertrap
    } else if((which_
        // ok
    } else {
        printf("usertra
        printf("
        panic("usertrap
    }

    if(which_dev == 2
        yield();

    prepare_return();

    uint64 atp = MAKE_SATP(p->pagetable);

    return satp;
}
```

内核虚拟地址空间

物理地址空间

进程虚拟地址空间

trampoline

uservec:

trampoline

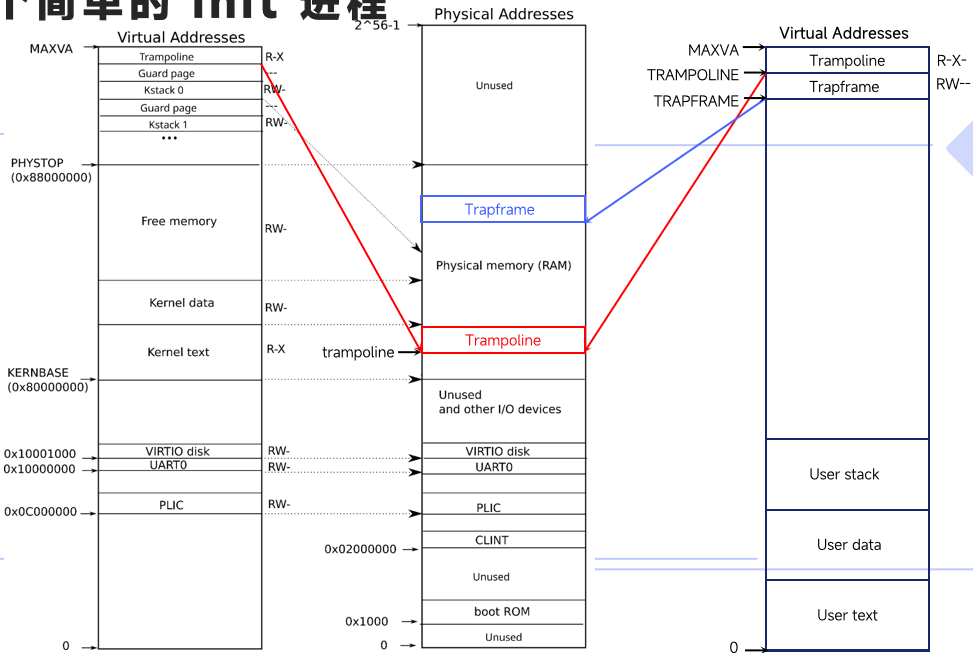
x pid=%d\n
uservec
ave not st



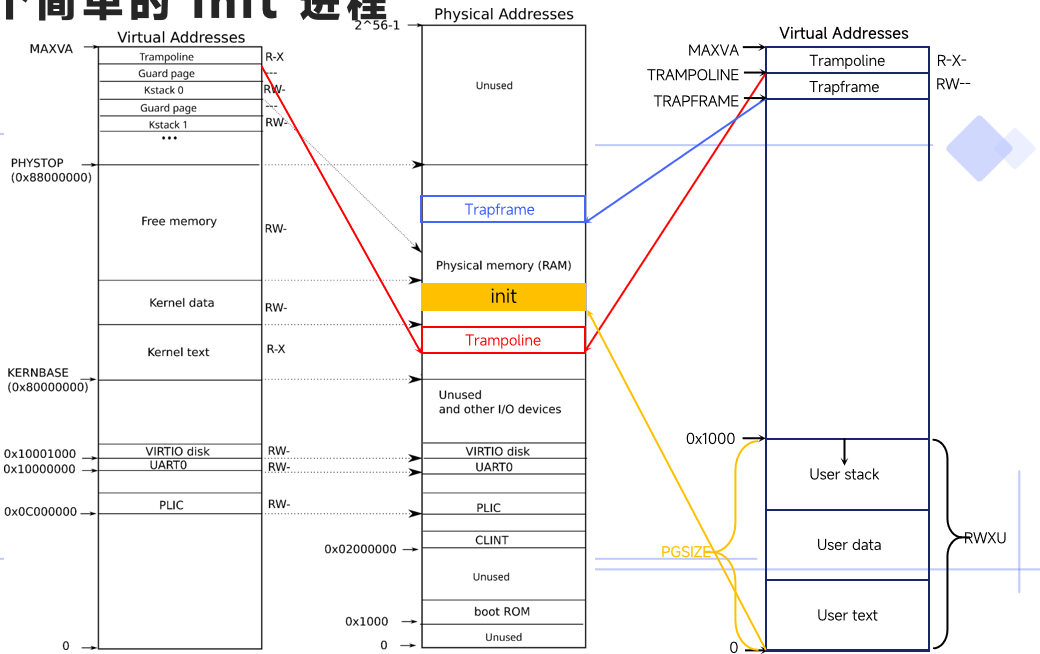
03

第一个用户态程序/进程

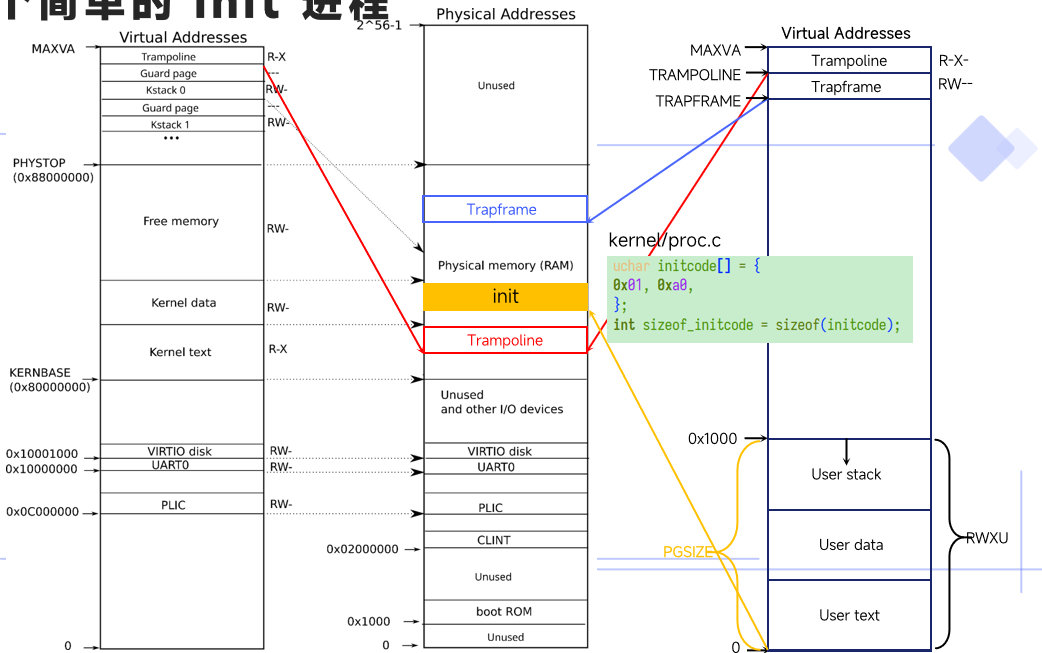
第一个简单的 init 进程



第一个简单的 init 进程



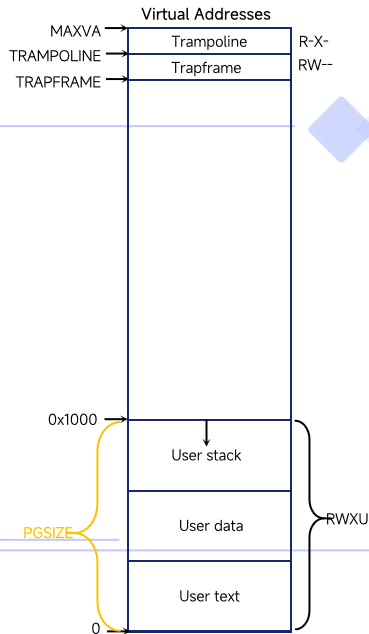
第一个简单的 init 进程



第一个简单的 init 进程

kernel/proc.c

```
uchar initcode[] = {  
    0x01, 0xa0,  
};  
int sizeof_initcode = sizeof(initcode);
```

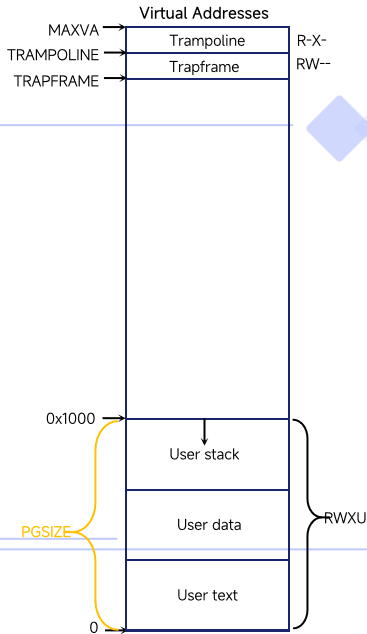


第一个简单的 init 进程 - 程序

kernel/proc.c

```
uchar initcode[] = {  
    0x01, 0xa0,  
};  
int sizeof_initcode = sizeof(initcode);
```

程序 (Program)



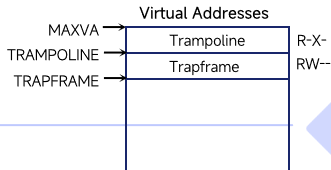
第一个简单的 init 进程 - 程序

kernel/proc.c

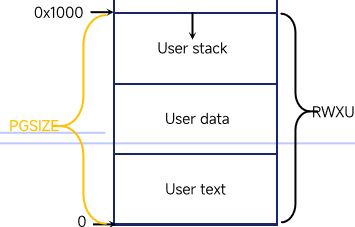
```
uchar initcode[] = {  
    0x01, 0xa0,  
};  
int sizeof_initcode = sizeof(initcode);
```

Init.S

```
.globl _start  
_start:  
_loop:  
    j _loop    # Infinite loop
```



```
$ riscv64-unknown-elf-gcc -c -o init.o init.S  
$ riscv64-unknown-elf-ld -N -e _start -Ttext 0 -o initcode.out init.o  
$ riscv64-unknown-elf-objcopy -S -O binary initcode.out initcode  
$ $ od -t xC initcode  
00000000 01 a0  
00000002
```



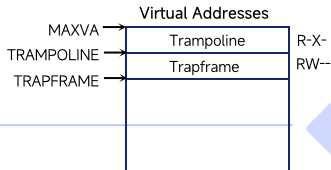
第一个简单的 init 进程 - 程序

kernel/proc.c

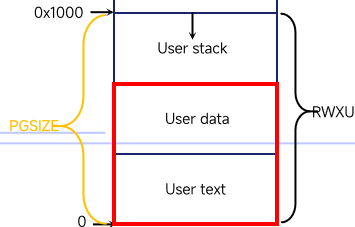
```
uchar initcode[] = {  
    0x01, 0xa0,  
};  
int sizeof_initcode = sizeof(initcode);
```

Init.S

```
.globl _start  
_start:  
_loop:  
    j _loop    # Infinite loop
```



```
$ riscv64-unknown-elf-gcc -c -o init.o init.S  
$ riscv64-unknown-elf-ld -N -e _start -Ttext 0 -o initcode.out init.o  
$ riscv64-unknown-elf-objcopy -S -O binary initcode.out initcode  
$ $ od -t xC initcode  
00000000 01 a0  
00000002
```



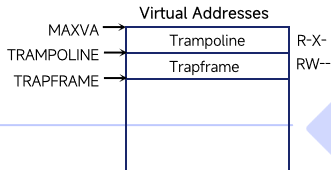
第一个简单的 init 进程 - 程序

kernel/proc.c

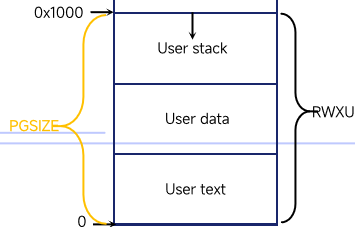
```
uchar initcode[] = {  
    0x01, 0xa0,  
};  
int sizeof_initcode = sizeof(initcode);
```

Init.S

```
.globl _start  
_start:  
_loop:  
    j _loop    # Infinite loop
```



```
$ riscv64-unknown-elf-gcc -c -o init.o init.S  
$ riscv64-unknown-elf-ld -N -e _start -Ttext 0 -o initcode.out init.o  
$ riscv64-unknown-elf-objcopy -S -O binary initcode.out initcode  
$ od -t xC initcode  
00000000 01 a0  
00000002
```



第一个简单的 init 进程 - 补全页表映射

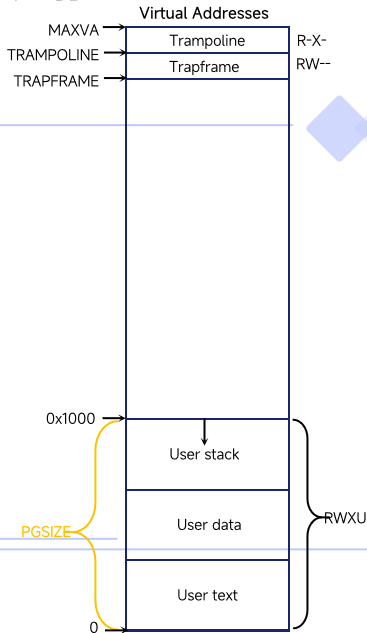
kernel/proc.c

```
uchar initcode[] = {  
    0x01, 0xa0,  
};  
int sizeof_initcode = sizeof(initcode);
```

程序 (Program)

```
static int loadinit(uchar *src, uint sz)  
{  
    char *mem;  
    struct proc *p = myproc();  
    pagetable_t pagetable = p->pagetable;  
  
    if(sz > PGSIZE)  
        panic("loadinit: more than a page");  
    mem = kalloc();  
    if(mem == 0)  
        panic("kalloc");  
    memset(mem, 0, PGSIZE);  
    if (mappages(pagetable, 0, PGSIZE, (uint64)mem, PTE_W | PTE_R | PTE_X | PTE_U) < 0)  
        panic("loadinit: mappages failed");  
    memmove(mem, src, sz);  
    p->sz = PGSIZE;  
  
    safestrcpy(p->name, "init", sizeof(p->name));  
  
    // prepare for the very first "return" from kernel to user.  
    p->trapframe->epc = 0;    // user program counter  
    p->trapframe->sp = PGSIZE; // user stack pointer  
  
    return 0;  
}
```

进程 (Process)



第一个简单的 init 进程 - 补全页表映射

kernel/proc.c

```
uchar initcode[] = {  
    0x01, 0xa0,  
};  
int sizeof_initcode = sizeof(initcode);
```

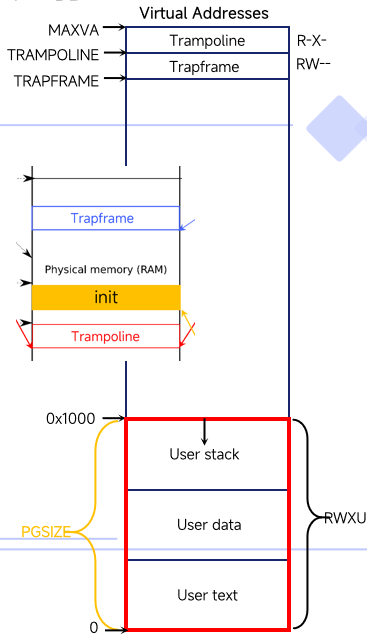
```
static int loadinit(uchar *src, uint sz)
```

```
{  
    char *mem;  
    struct proc *p = myproc();  
    pagetable_t pagetable = p->pagetable;
```

```
    if (sz > PGSIZE)  
        panic("loadinit: more than a page");  
    mem = kalloc();  
    if (mem == 0)  
        panic("kalloc");  
    memset(mem, 0, PGSIZE);  
    if (mappages(pagetable, 0, PGSIZE, (uint64)mem, PTE_W | PTE_R | PTE_X | PTE_U) < 0)  
        panic("loadinit: mappages failed");  
    memmove(mem, src, sz);  
    p->sz = PGSIZE;  
  
    safestrcpy(p->name, "init", sizeof(p->name));
```

```
    // prepare for the very first "return" from kernel to user.  
    p->trapframe->epc = 0;    // user program counter  
    p->trapframe->sp = PGSIZE; // user stack pointer
```

```
    return 0;  
}
```



第一个简单的 init 进程 - 补全页表映射

kernel/proc.c

```
uchar initcode[] = {
    0x01, 0xa0,
};
int sizeof_initcode = sizeof(initcode);

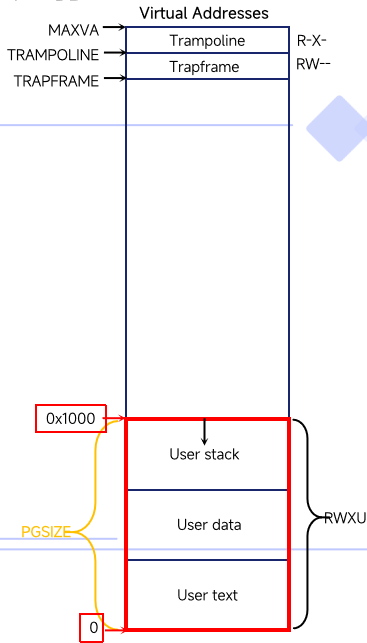
static int loadinit(uchar *src, uint sz)
{
    char *mem;
    struct proc *p = myproc();
    pagetable_t pagetable = p->pagetable;

    if (sz > PGSIZE)
        panic("loadinit: more than a page");
    mem = kalloc();
    if (mem == 0)
        panic("kalloc");
    memset(mem, 0, PGSIZE);
    if (mappages(pagetable, 0, PGSIZE, (uint64)mem, PTE_W | PTE_R | PTE_X | PTE_U) < 0)
        panic("loadinit: mappages failed");
    memmove(mem, src, sz);
    p->sz = PGSIZE;

    safestrcpy(p->name, "init", sizeof(p->name));

    // prepare for the very first "return" from kernel to user.
    p->trapframe->epc = 0;    // user program counter
    p->trapframe->sp = PGSIZE; // user stack pointer

    return 0;
}
```



第一个简单的 init 进程 - 启动进入用户态

kernel/proc.c

```
// Set up first user process.  
void userinit(void)  
{  
    struct proc *p;  
  
    p = allocproc();  
    initproc = p;  
  
    p->state = RUNNABLE;  
  
    release(&p->lock);  
}
```



第一个简单的 init 进程 - 启动进入用户态

kernel/proc.c

```
// Set up first user
void userinit(void)
{
    struct proc *p;

    p = allocproc();
    initproc = p;

    p->state = RUNNABLE;
    release(&p->lock);
}
```

```
diff --git a/kernel/proc.c b/kernel/proc.c
index 9fb63dc..d2fe915 100644
--- a/kernel/proc.c
+++ b/kernel/proc.c
@@ -131,13 +133,12 @@ found:
     return 0;
 }

- // Set up new context to start executing
+ // Set up new context to start executing at forkret,
+ // which returns to user space.
    memset(&p->context, 0, sizeof(p->context));
- p->context.ra = (uint64)kerneltrapret;
+ p->context.ra = (uint64)forkret;
    p->context.sp = p->kstack + PGSIZE;

- p->start = start_routine;
-
    return p;
}
```

第一个简单的 init 进程 - 启动进入用户态

kernel/proc.c

```
// Set up first user
void userinit(void)
{
    struct proc *p;

    p = allocproc();
    initproc = p;

    p->state = RUNNABLE;
    release(&p->lock);
}
```

```
diff --git a/kernel/proc.c b/kernel/proc.c
index 9fb63dc..d2fe915 100644
--- a/kernel/proc.c
+++ b/kernel/proc.c
@@ -131,13 +133,12 @@ found:
     return 0;
 }

- // Set up new context to start executing
+ // Set up new context to start executing
+ // which returns to user space.
memset(&p->context, 0, sizeof(p->context));
- p->context.ra = (uint64)kerneltrapret;
+ p->context.ra = (uint64)forkret;
p->context.sp = p->kstack + PGSIZE;

p->start = start_routine;
return p;
}
```

```
void forkret(void)
{
    extern char userret[];
    static int first = 1;
    struct proc *p = myproc();

    // Still holding p->lock from scheduler.
    release(&p->lock);

    if (first) {
        first = 0;
        // ensure other cores see first=0.
        __sync_synchronize();

        // We can invoke loadinit() now.
        // Put the return value (argc) of loadinit into a0.
        p->trapframe->a0 = loadinit(initcode, sizeof_initcode);
        if (p->trapframe->a0 == -1) {
            panic("loadinit");
        }
    }

    // return to user space, mimicing usertrap()'s return.
    prepare_return();
    uint64 satp = MAKE_SATP(p->pagetable);
    uint64 trampoline_userret = TRAMPOLINE + (userret - trampoline);
    ((void (*)(uint64))trampoline_userret)(satp);
}
```

第一个简单的 init 进程 - 启动进入用户态

kernel/proc.c

```
// Set up first user
void userinit(void)
{
    struct proc *p;

    p = allocproc();
    initproc = p;

    p->state = RUNNABLE;
    release(&p->lock);
}
```

```
diff --git a/kernel/proc.c b/kernel/proc.c
index 9fb63dc..d2fe915 100644
--- a/kernel/proc.c
+++ b/kernel/proc.c
static struct proc*
-allocproc(void (*start_routine)(void))
+allocproc(void)
{
    struct proc *p;

    void scheduler(void)
    {
        // .....
        switch(&c->context, &p->context);
        // .....
    }
```

```
memset(&p->context, 0, sizeof(p->context));
- p->context.ra = (uint64)kerneltrapret;
+ p->context.ra = (uint64)forkret;
p->context.sp = p->kstack + PGSIZE;

- p->start = start_routine;
-
    return p;
}
```

```
void forkret(void)
{
    extern char userret[];
    static int first = 1;
    struct proc *p = myproc();

    // Still holding p->lock from scheduler.
    release(&p->lock);

    if (first) {
        first = 0;
        // ensure other cores see first=0.
        __sync_synchronize();

        // We can invoke loadinit() now.
        // Put the return value (argc) of loadinit into a0.
        p->trapframe->a0 = loadinit(initcode, sizeof_initcode);
        if (p->trapframe->a0 == -1) {
            panic("loadinit");
        }
    }

    // return to user space, mimicing usertrap()'s return.
    prepare_return();
    uint64 satp = MAKE_SATP(p->pagetable);
    uint64 trampoline_userret = TRAMPOLINE + (userret - trampoline);
    ((void (*)(uint64))trampoline_userret)(satp);
}
```

第一个简单的 init 进程 - 启动进入用户态

kernel/proc.c

```
// Set up first user
void userinit(void)
{
    struct proc *p;

    p = allocproc();
    initproc = p;

    p->state = RUNNABLE;
    release(&p->lock);
}
```

```
diff --git a/kernel/proc.c b/kernel/proc.c
index 9fb63dc..d2fe915 100644
--- a/kernel/proc.c
+++ b/kernel/proc.c
static struct proc*
-allocproc(void (*start_routine)(void))
+allocproc(void)
{
    struct proc *p;

    void scheduler(void)
    {
        // .....
        switch(&c->context, &p->context);
        // .....
    }
```

```
memset(&p->context, 0, sizeof(p->context));
- p->context.ra = (uint64)kerneltrapret;
+ p->context.ra = (uint64)forkret;
p->context.sp = p->kstack + PGSIZE;

- p->start = start_routine;
-
    return p;
}
```

void forkret(void)

```
{
    extern char userret[];
    static int first = 1;
    struct proc *p = myproc();

    // Still holding p->lock from scheduler.
    release(&p->lock);
```

```
if (first) {
    first = 0;
    // ensure other
    __sync_synchronize();

    // We can invoke
    // Put the return
    p->trapframe->a0
    if (p->trapframe
        panic("loadin
    }
```

```
// return to user
prepare_return();
uint64 satp = MAKE
uint64 trampoline_
((void (*)(uint64)
```

void prepare_return(void)

```
{
    struct proc *p = myproc();

    intr_off();

    uint64 trampoline_uservec = TRAMPOLINE + (uservec - trampoline);
    w_stvec(trampoline_uservec);

    p->trapframe->kernel_satp = r_satp(); // kernel page table
    p->trapframe->kernel_sp = p->kstack + PGSIZE; // process's kernel stack
    p->trapframe->kernel_trap = (uint64)usertrap;
    p->trapframe->kernel_hartid = r_tp(); // hartid for cpuid()

    unsigned long x = r_sstatus();
    x &= ~SSTATUS_SPP; // clear SPP to 0 for user mode
    x |= SSTATUS_SPIE; // enable interrupts in user mode
    w_sstatus(x);

    w_sepc(p->trapframe->epc);
}
```

第一个简单的 init 进程 - 启动进入用户态

kernel/proc.c

```
// Set up first user
void userinit(void)
{
    struct proc *p;

    p = allocproc();
    initproc = p;

    p->state = RUNNABLE;
    release(&p->lock);
}
```

```
diff --git a/kernel/proc.c b/kernel/proc.c
index 9fb63dc..d2fe915 100644
--- a/kernel/proc.c
+++ b/kernel/proc.c
static struct proc*
-allocproc(void (*start_routine)(void))
+allocproc(void)
{
    struct proc *p;

    void scheduler(void)
    {
        // .....
        switch(&c->context, &p->context);
        // .....
    }
```

```
memset(&p->context, 0, sizeof(p->context));
- p->context.ra = (uint64)kerneltrapret;
+ p->context.ra = (uint64)forkret;
p->context.sp = p->kstack + PGSIZE;

- p->start = start_routine;
-
    return p;
}
```

void forkret(void)

```
{
    extern char userret[];
    static int first = 1;
    struct proc *p = myproc();

    // Still holding p->lock from scheduler.
    release(&p->lock);
```

```
if (first) {
    first = 0;
    // ensure other
    __sync_synchronize();

    // We can invoke
    // Put the return
    p->trapframe->a0
    if (p->trapframe
        panic("loadini
    }
```

```
// return to user
prepare_return();
uint64 satp = MAKE
uint64 trampoline_
((void (*)(uint64)
```

void prepare_return(void)

```
{
    struct proc *p = myproc();

    intr_off();

    uint64 trampoline_user
    w_stvec(trampoline_user

    p->trapframe->kernel_s
    p->trapframe->kernel_s
    p->trapframe->kernel_t
    p->trapframe->kernel_h
```

```
unsigned long x = r_s
x &= ~SSTATUS_SPP; //
x |= SSTATUS_SPIE; //
w_sstatus(x);
```

w_sepc(p->trapframe->epc);

static int loadinit(uchar *src, uint sz)

```
{
    char *mem;
    struct proc *p = myproc();
    // .....
    // prepare for the very first "return" from kernel to user.
    p->trapframe->epc = 0; // user program counter
    p->trapframe->sp = PGSIZE; // user stack pointer

    return 0;
}
```


第一个简单的 init 进程 - 启动进入用户态

kernel/proc.c

```
// Set up first user
void userinit(void)
{
    struct proc *p;

    p = allocproc();
    initproc = p;

    p->state = RUNNABLE;
    release(&p->lock);
}
```

diff --git a/kernel/proc.c b/kernel/proc.c
index 9fb63dc..d2fe915 100644
--- a/kernel/proc.c
+++ b/kernel/proc.c

```
static struct proc*  
-allocproc(void (*start_routine)(void))  
+allocproc(void)  
{  
    struct proc *p;
```

void scheduler(void)

```
{  
    // .....  
    switch(&c->context, &p->context);  
    // .....  
}
```

```
memset(&p->context, 0, sizeof(p->context));  
- p->context.ra = (uint64)kerneltrapret;  
+ p->context.ra = (uint64)forkret;  
p->context.sp = p->kstack + PGSIZE;  
  
- p->start = start_routine;  
-  
    return p;  
}
```

void forkret(void)

```
{  
    extern char userret[];  
    static int first = 1;  
    struct proc *p = myproc();  
  
    // Still holding p->lock from scheduler.  
    release(&p->lock);  
  
    if (first) {  
        first = 0;  
        // ensure other cores see first=0.  
        __sync_synchronize();  
  
        // We can invoke loadinit() now.  
        // Put the return value (argc) of loadinit into a0.  
        p->trapframe->a0 = loadinit(initcode, sizeof_initcode);  
        if (p->trapframe->a0 == -1) {  
            panic("loadinit");  
        }  
    }  
  
    // return to user space, mimicing usertrap()'s return.  
    prepare_return();  
    uint64 satp = MAKE_SATP(p->pagetable);  
    uint64 trampoline_userret = TRAMPOLINE + (userret - trampoline);  
    ((void (*)(uint64))trampoline_userret)(satp);  
}
```

kernel/trampoline.S

```
uservec:  
    csrw sscratch, a0  
    li a0, TRAPFRAME  
    sd ra, 40(a0)  
    .....  
    sd t6, 280(a0)  
    csrr t0, sscratch  
    sd t0, 112(a0)  
  
    ld sp, 8(a0)  
    ld tp, 32(a0)  
    ld t0, 16(a0)  
    ld t1, 0(a0)  
  
    sfence.vma zero, zero  
    csrw satp, t1  
    sfence.vma zero, zero  
  
    jalr t0
```

```
userret:  
    sfence.vma zero, zero  
    csrw satp, a0  
    sfence.vma zero, zero  
  
    li a0, TRAPFRAME  
    ld ra, 40(a0)  
    .....  
    ld t6, 280(a0)  
    ld a0, 112(a0)  
  
    sret
```

第一个简单的 init 进程 - 启动进入用户态

kernel/proc.c

```
// Set up first user
void userinit(void)
{
    struct proc *p;
    p = allocproc();
    initproc = p;

    p->state = RUNNABLE;
    release(&p->lock);
}
```

```
diff --git a/kernel/proc.c b/kernel/proc.c
index 9fb63dc..d2fe915 100644
--- a/kernel/proc.c
+++ b/kernel/proc.c
```

```
static struct proc*
-allocproc(void (*start_routine)(void))
+allocproc(void)
{
    struct proc *p;
```

```
void scheduler(void)
{
    // .....
    switch(&c->context, &p->context);
    // .....
}
```

```
memset(&p->context, 0, sizeof(p->context));
- p->context.ra = (uint64)kerneltrapret;
+ p->context.ra = (uint64)forkret;
```

void forkret(void)

```
{
    extern char userret[];
    static int first = 1;
    struct proc *p = myproc();

    // Still holding p->lock from scheduler.
    release(&p->lock);

    if (first) {
        first = 0;
        // ensure other cores see first=0.
        __sync_synchronize();

        // We can invoke loadinit() now.
        // Put the return value (argc) of loadinit into a0.
        p->trapframe->a0 = loadinit(initcode, sizeof_initcode);
    }
```

```
$make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic
```

xv6 kernel is booting

hart 2 starting
hart 1 starting

1 run init

按 Ctrl + P

kernel/trampoline.S

```
uservec:
    csrw sscratch, a0
    li a0, TRAPFRAME
    sd ra, 40(a0)
    .....
    sd t6, 280(a0)
    csrr t0, sscratch
    sd t0, 112(a0)

    ld sp, 8(a0)
    ld tp, 32(a0)
    ld t0, 16(a0)
    ld t1, 0(a0)

    sfence.vma zero, zero
    csrw satp, t1
    sfence.vma zero, zero

    jalr t0
```

```
userret:
    sfence.vma zero, zero
    csrw satp, a0
    sfence.vma zero, zero

    li a0, TRAPFRAME
    ld ra, 40(a0)
    .....
    ld t6, 280(a0)
    ld a0, 112(a0)

    sret
```



04

改进程序构建

目前程序的构建方式

kernel/proc.c

```
// a very simple user program 'init.S':  
// .globl _start  
// _start:  
// _loop:  
//     j _loop # Infinite loop  
//  
// initcode[] is generated by:  
// riscv64-unknown-elf-gcc -c -o init.o init.S  
// riscv64-unknown-elf-ld -N -e _start -Ttext 0 -o initcode.out init.o  
// riscv64-unknown-elf-objcopy -S -O binary initcode.out initcode  
// od -t xC initcode  
uchar initcode[] = {  
    0x01, 0xa0,  
};  
int sizeof_initcode = sizeof(initcode);
```

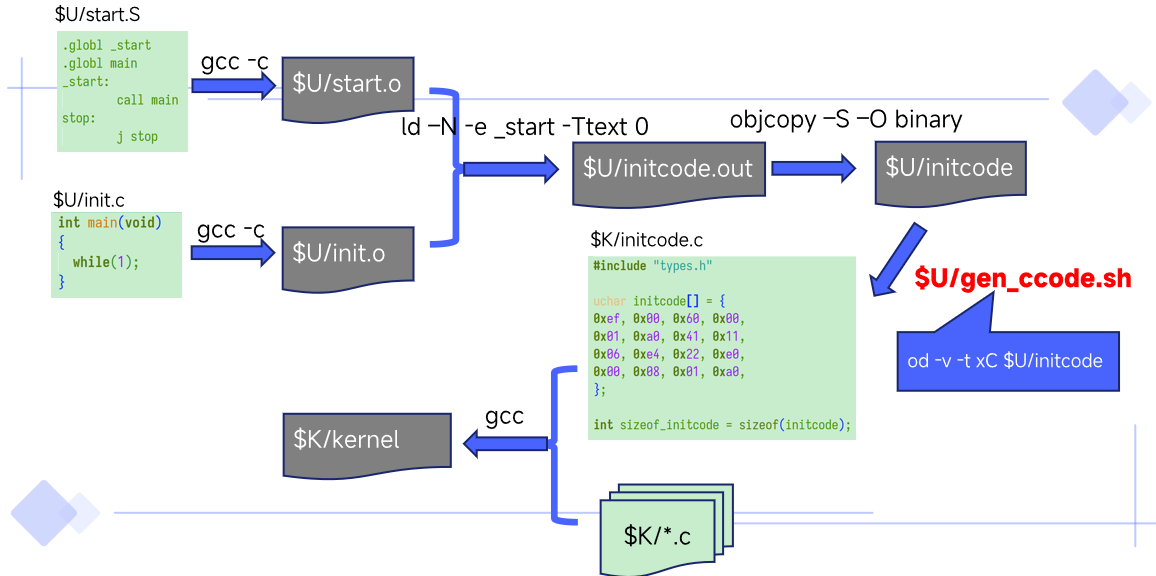
目前程序的构建方式

kernel/proc.c

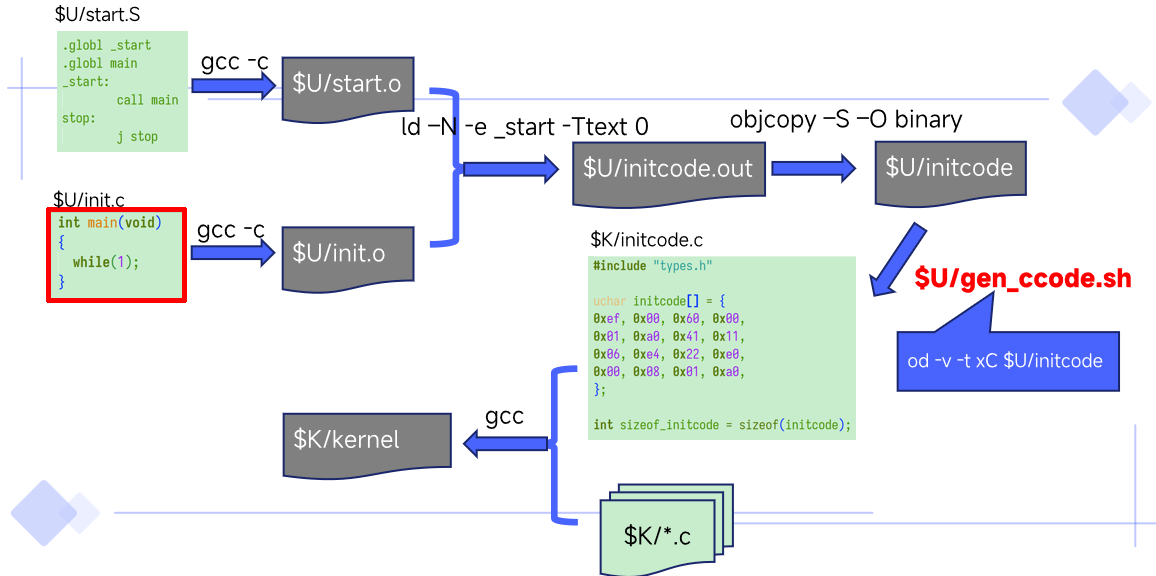
```
// a very simple user program 'init.S':  
// .globl _start  
// _start:  
// _loop:  
//     j _loop # Infinite loop  
//  
// initcode[] is generated by:  
// riscv64-unknown-elf-gcc -c -o init.o init.S  
// riscv64-unknown-elf-ld -N -e _start -Ttext 0 -o initcode.out init.o  
// riscv64-unknown-elf-objcopy -S -O binary initcode.out initcode  
// od -t xC initcode  
uchar initcode[] = {  
    0x01, 0xa0,  
};  
int sizeof_initcode = sizeof(initcode);
```

手动介入太多
不方便扩展

改进后的程序构建方式

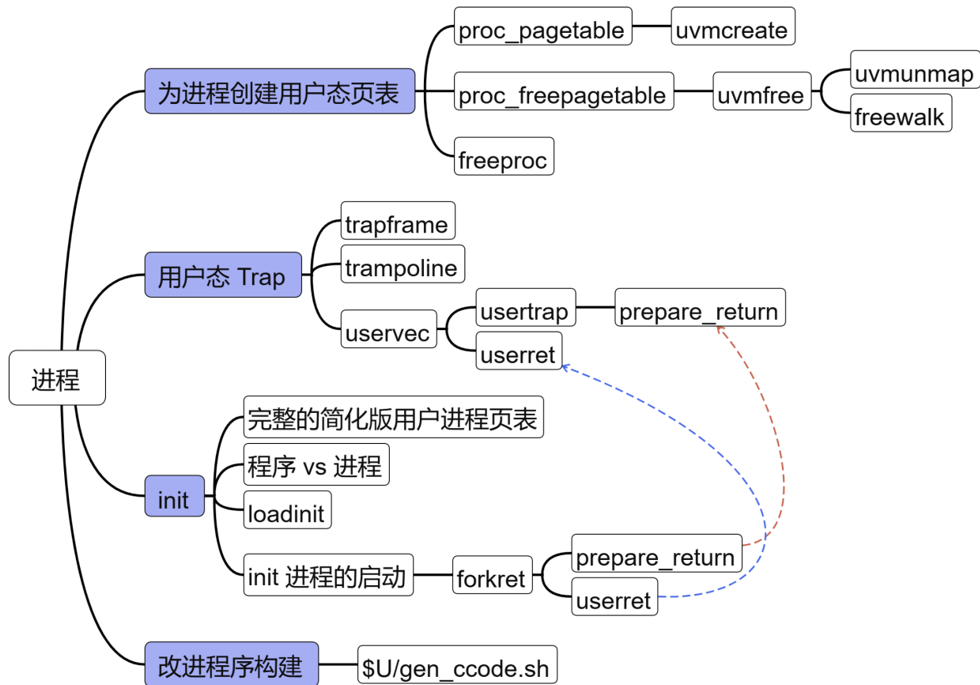


改进后的程序构建方式





本章总结



谢谢

